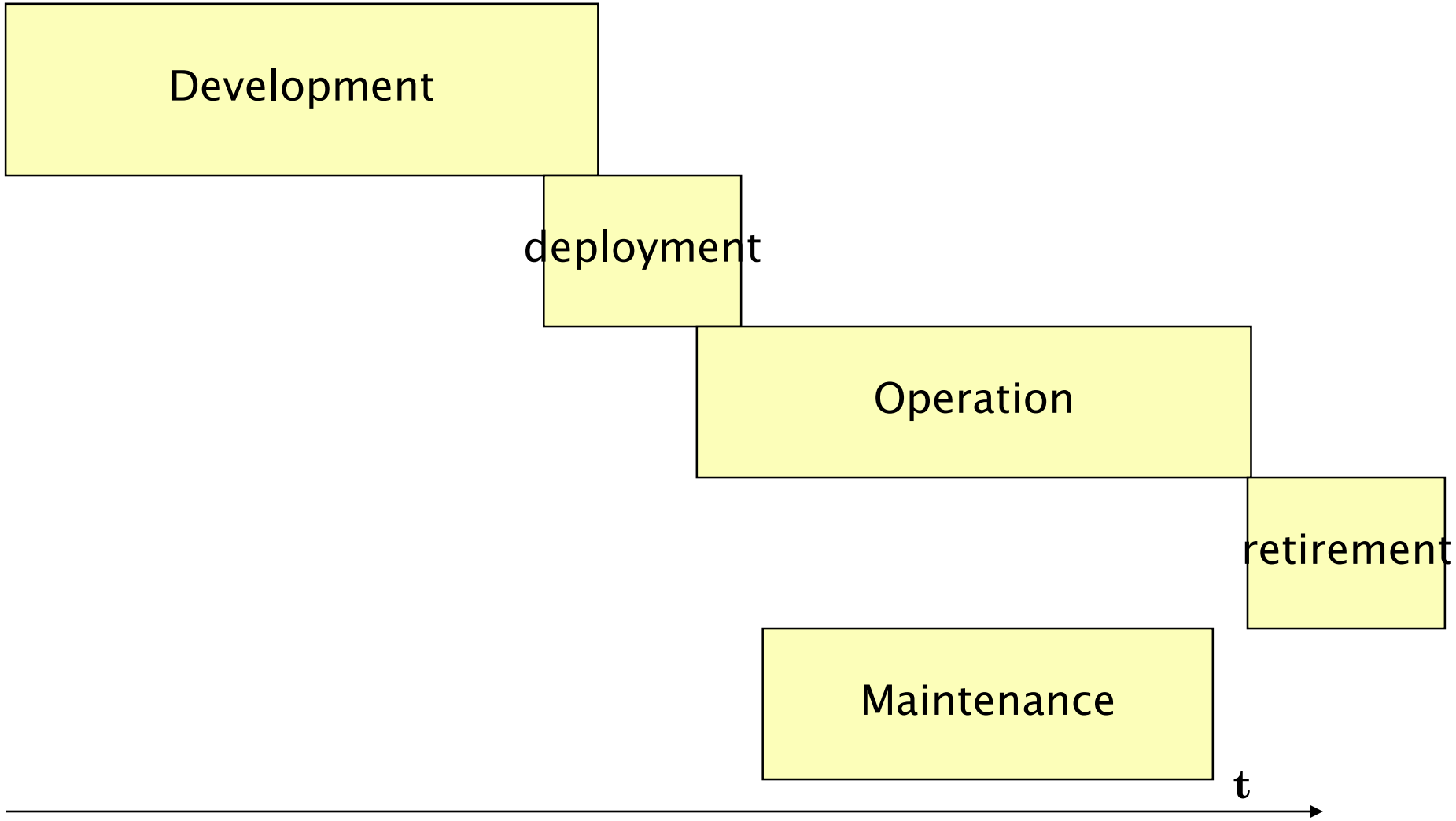
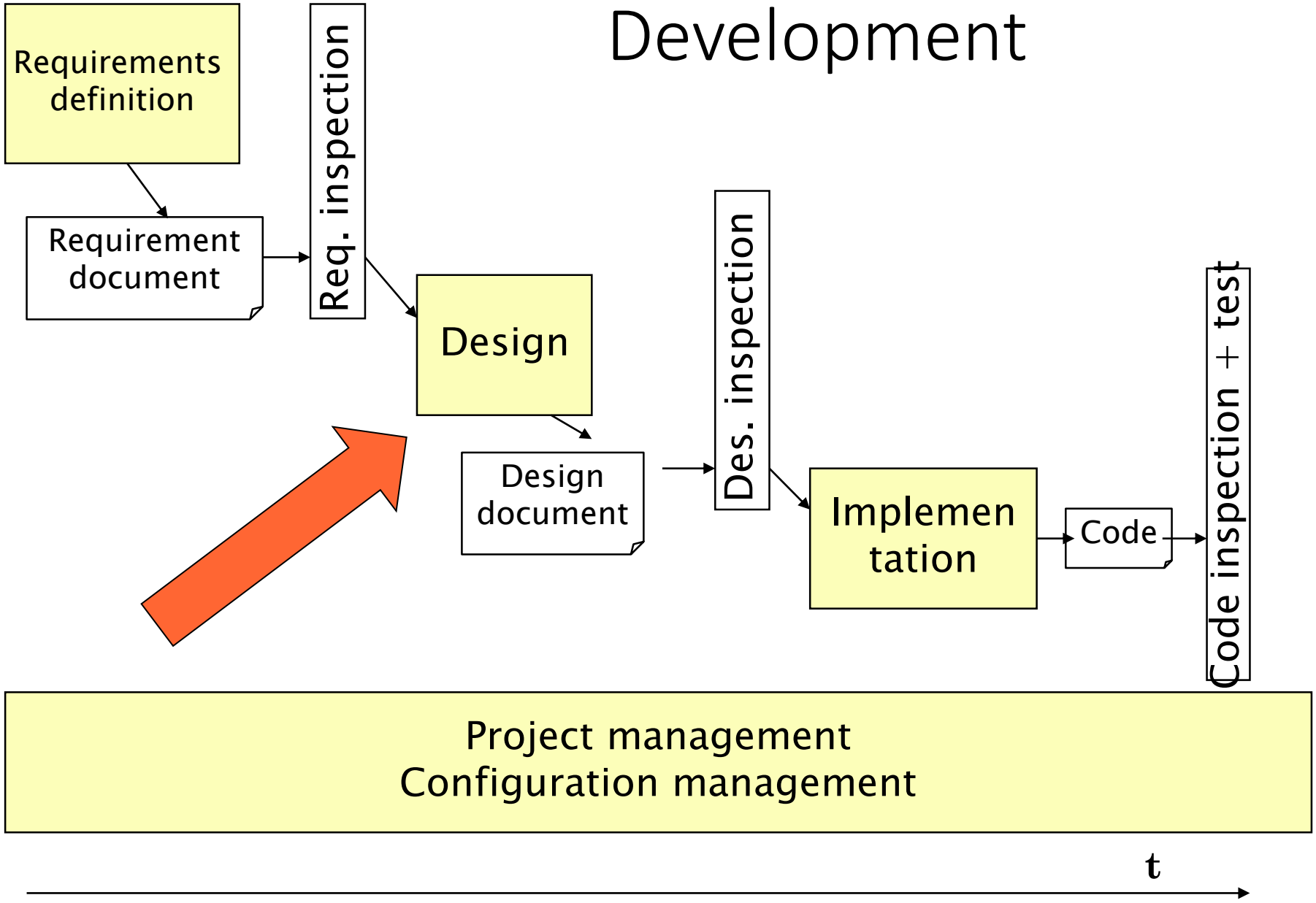
Architecture and Design

Main Phases



Development



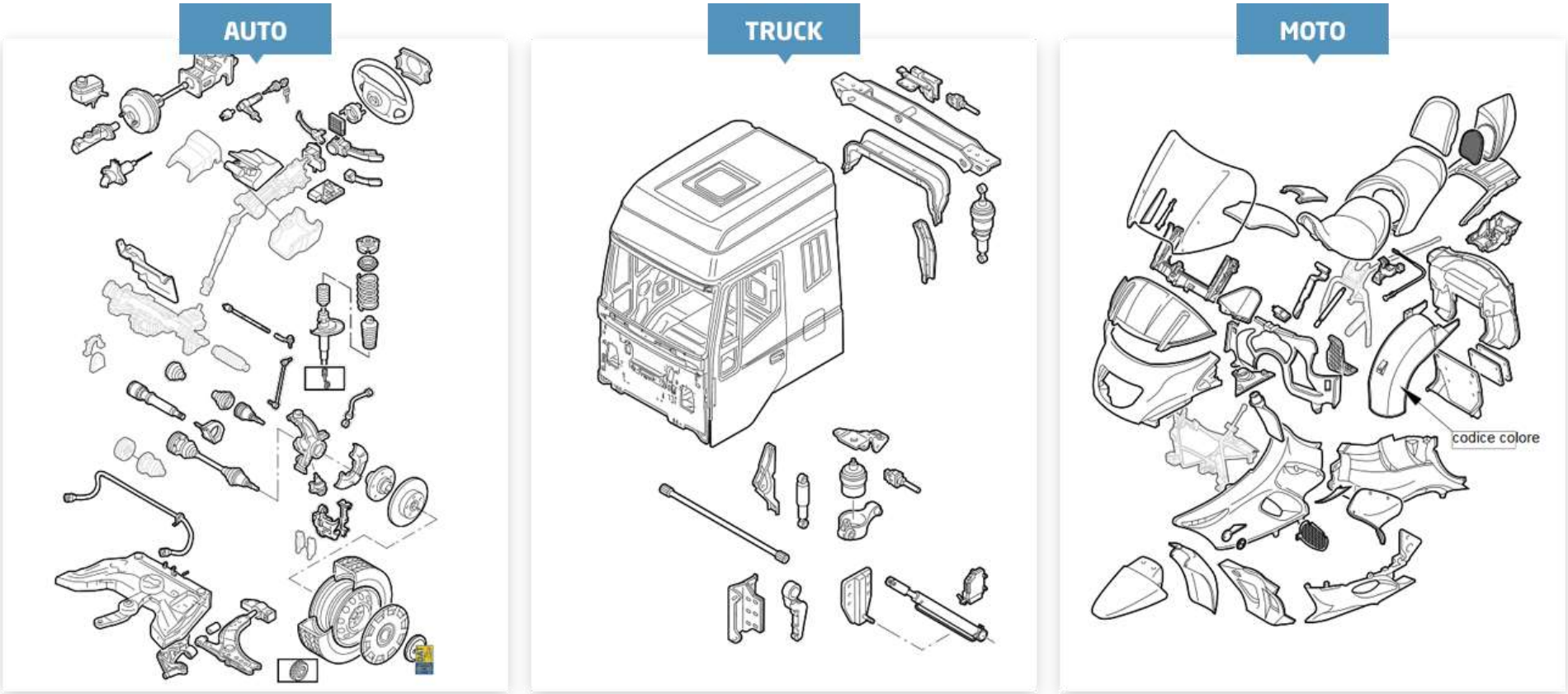
Outline

- Process
- Properties
- Notations
- Patterns
  - Architectural patterns /styles
  - Design patterns

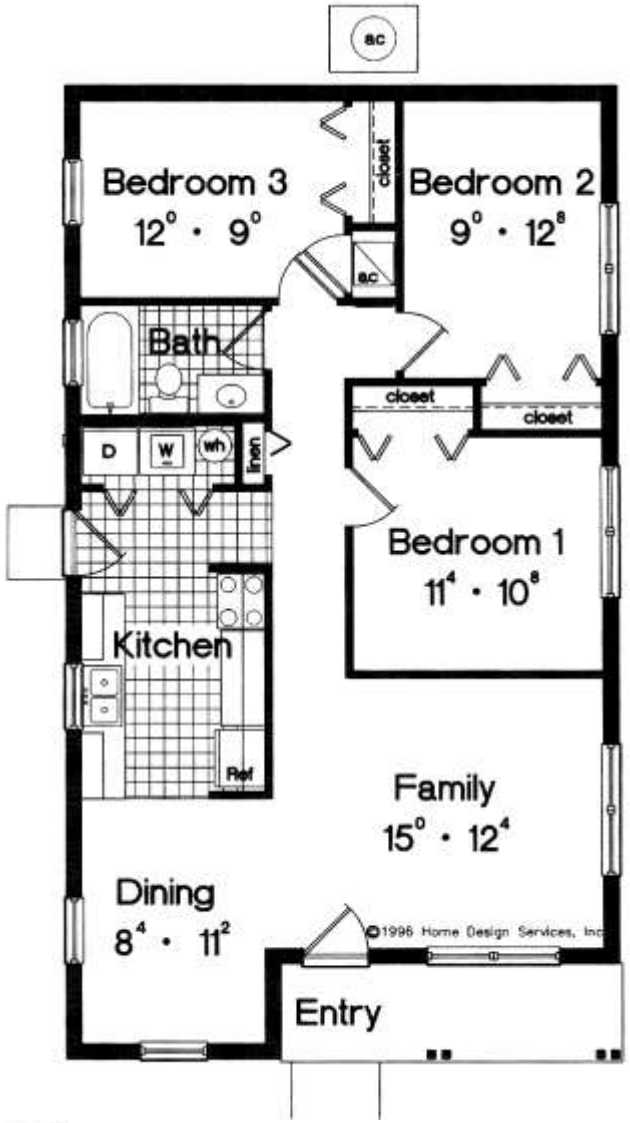
Architecture

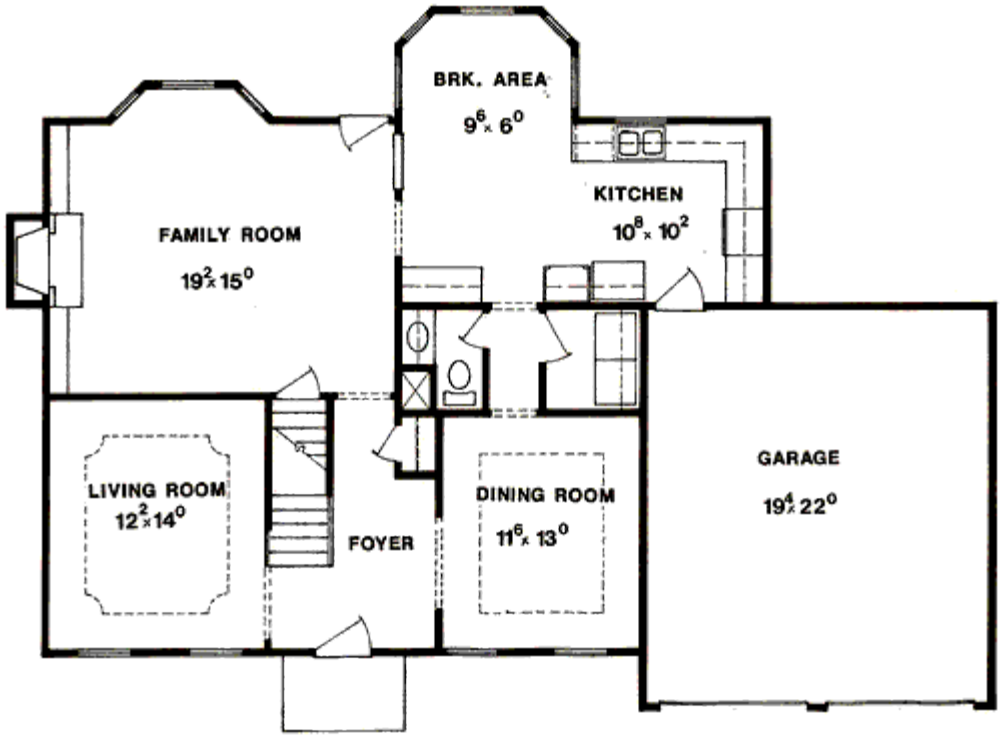
- Requirements: **what** the system should do
- Architecture, design: **how** the system should be built
  - Architecture, design: same flavour but
    - Architecture: high level, decide major components and their control and communication framework
    - Design: lower level, decide internals of each component

Vehicles



Houses





Architecture, design, why

- Most defects come from requirements and design
- Essential to define, analyze and evaluate design choices *early*
- If no design is defined, but code is developed immediately, design choices are made implicitly and evaluated *late*
- Doing design allows to make design choices *explicit* , document and evaluate them

Example

- Design choice: Access to db
  - Option1, SQL queries written everywhere
  - Option2, DAO objects only contain data access code

- What if database changes?
  - Ex. From my sql do sql lite
  - Ex. From local db to cloud service

  - Option1: need to change code everywhere
  - Option2: need to change DAO objects only

Requirements to design

- Given one set of requirements
- In general, many different designs are possible (*design choices*)
  - Cfr. Requirement: mid-sized car in the price range of 10 to 20k
  - Designs: hundreds of models on the market,
    - High-level design choices
      - diesel or gas or electric engine
      - front or rear or all-wheel drive
    - Low-level design choices
      - Colour, outer details
      - With ABS, ESP, .. or not
- But not all designs are equal

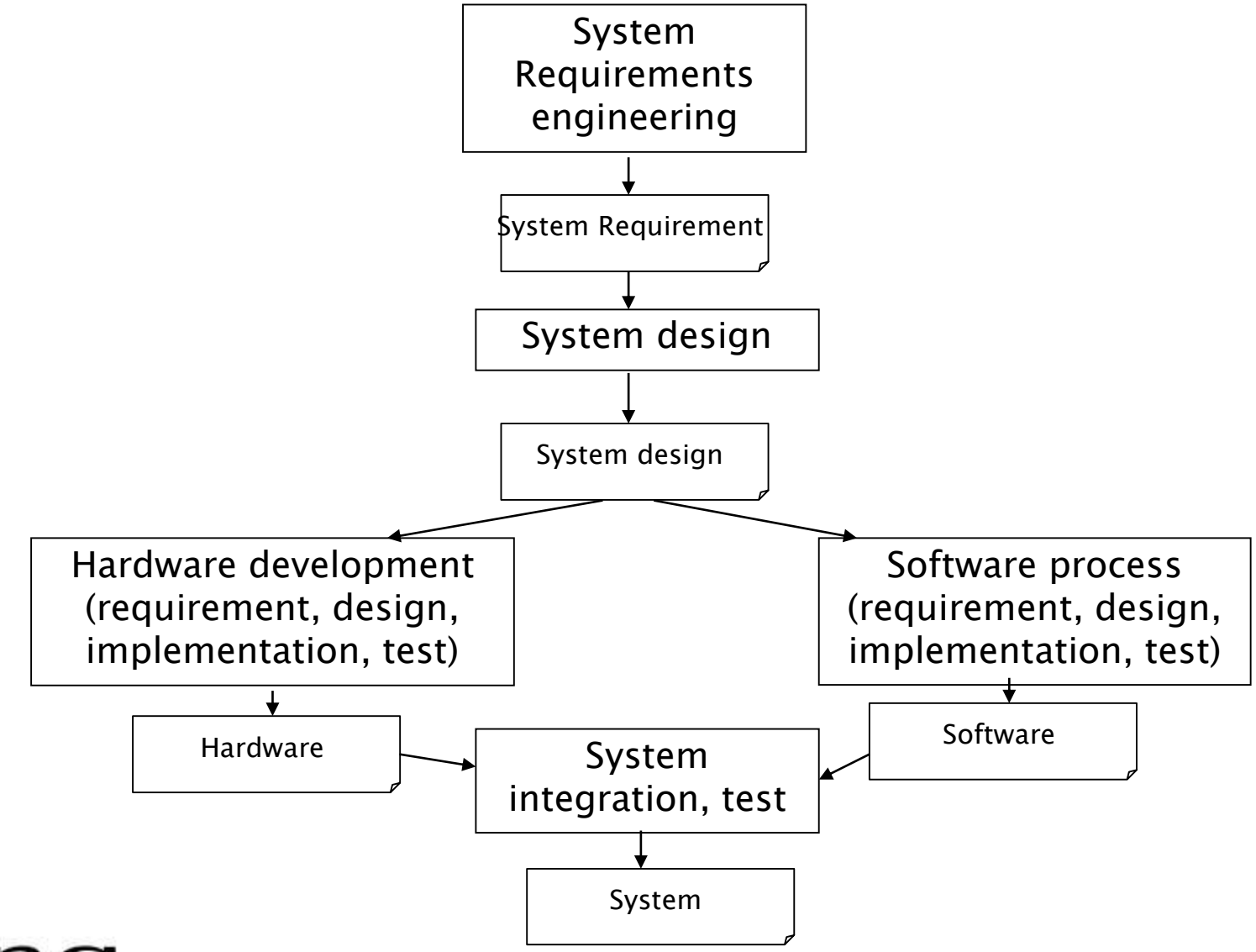
Requirements to design

- A creative process
- Driven by skill and experience
- Experience formalized in semi formal guidelines
  - Architectural styles (patterns)
  - Design patterns

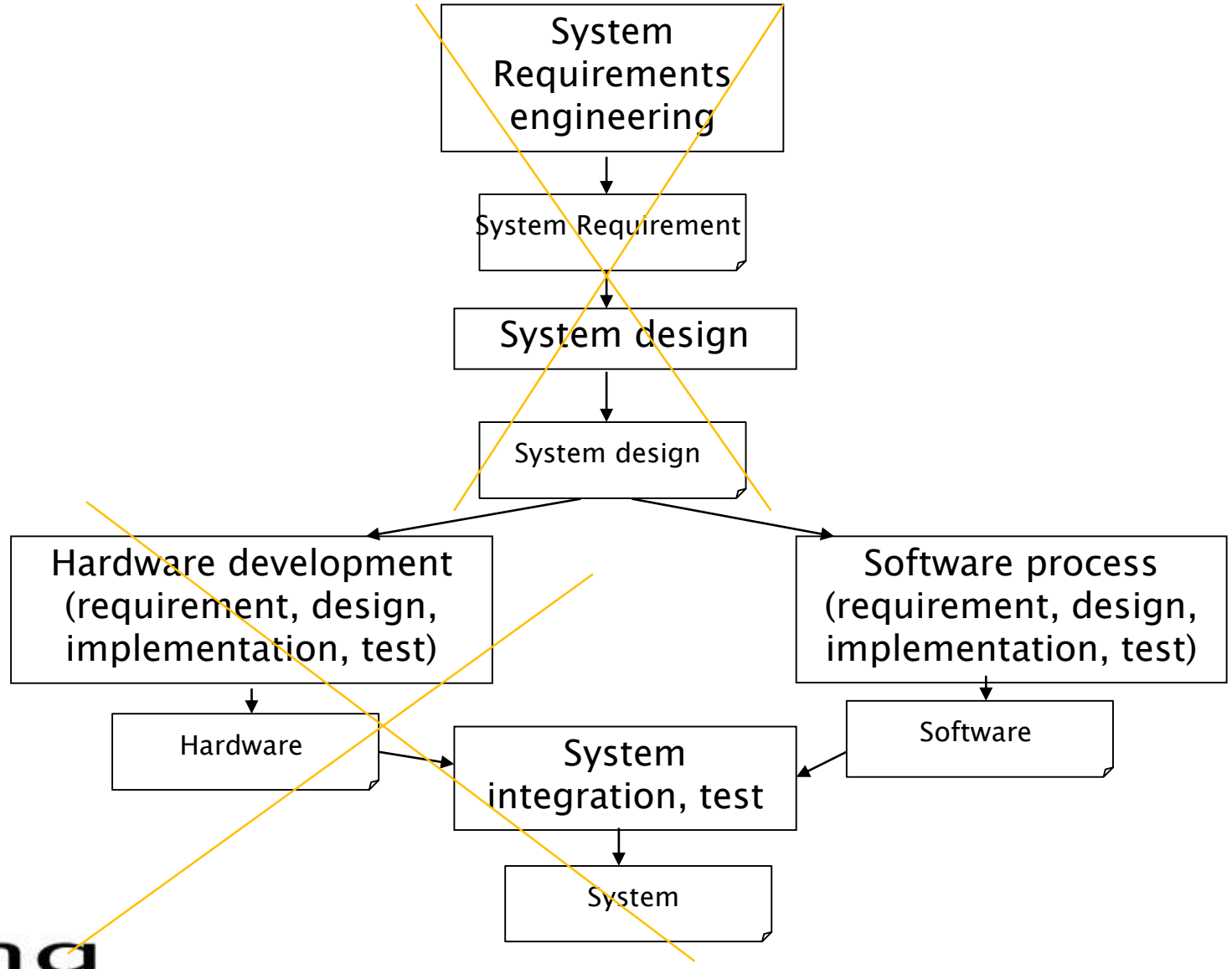
Two process variants

Software only process	System process
• (Software) Requirements • (Software) Design	• System requirements • System design • Software requirements • Software design

System process



Software process



Hw – sw interaction

- Software process includes also a part about hardware components and software – hardware allocation (UML Deployment diagram)

Example

- Heating control system, two cases of hw-sw allocation
  - Option1: One processor per room, a processor to control rooms and house
  - Option2: One processor per house

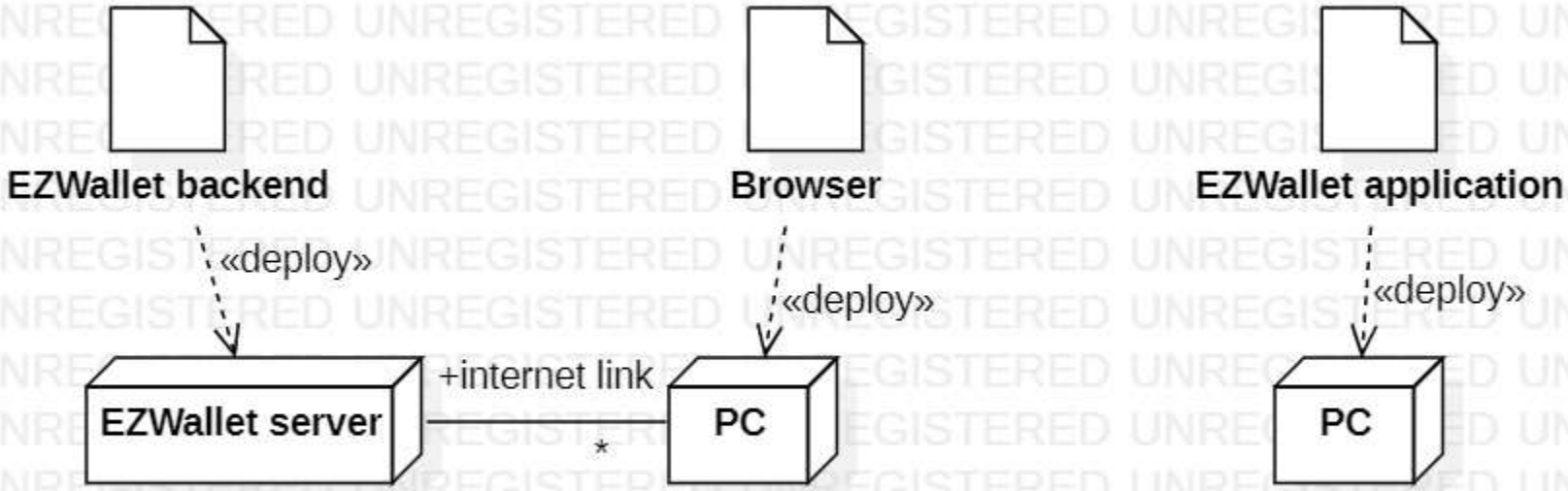
  - Option1: more expensive, more complex, more flexible, faster response
  - Option2: more cheap, simpler, less flexible, slower response

Example

- Two options of hw –sw allocation for EZGas

client-server
(multiple concurrent users)

single app
(1 user at a time)



Software design

- Having defined the hardware context, software design is about:
- Defining software modules (functions, classes, packages, modules, ..) and their interactions so that they satisfy:
  - functional
  - non - functional requirements

Process

Design Process

- Analysis
  - Architecture
  - High level design
  - Low level design
- Formalization
  - Text, diagrams (UML)
- Verification

Process – input and output

- Input
  - Requirement document

    (functional requirements

    non functional requirements)
- Output
  - Design document
    - Components + connections
    - Capable of satisfying functional + non functional requirements

Process - activities

- 1 Architecture

  (about the whole system)

  - Define high level components and their interactions
  - Select communication and coordination model
    - Processes, threads
    - Messages, (remote) procedure calls, broadcast, blackboard
  - Use architectural style(s) /pattern(s)

Process - activities

- 2 Design
  - 2.1 High level (about many classes)
    - Define classes and their interactions
    - Use design patterns
  - 2.2 Low level (about one class)

Properties

Properties of a design

- Functional properties
  - Does the design support the functional requirements?
    - Functional requirements (requirements document)

      vs.
    - functional properties (design)
- Non-functional properties
  - Does the design support the non-functional requirements?
    - Non-functional requirements (requirements document)

      vs.
    - Non-functional properties (design)

Non functional properties

- Reliability
- Efficiency/performance
- Usability
- Maintainability
- Portability
- Safety
- Security

Non functional properties

- More specific to design
  - Testability
    - observability
    - controllability
  - Monitorability
  - Interoperability
  - Scalability
  - Deployability
  - Mobility

Non functional properties

- Complexity
  - Number of components
  - Number of interactions
  - KISS: keep it simple, stupid
- Coupling (or decoupling)
  - Degree of dependence between two components
- Cohesion
  - Degree of consistence of functions of a component

Coupling

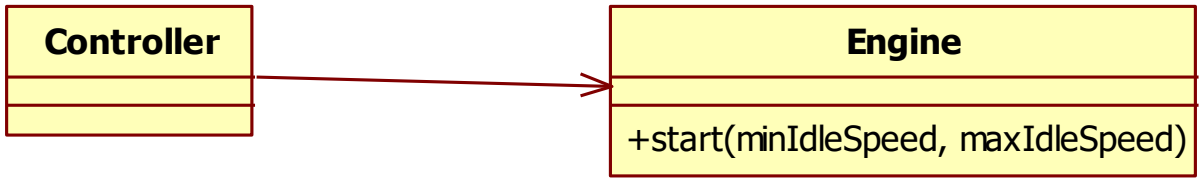
- Walls vs plumbing components

lower

higher

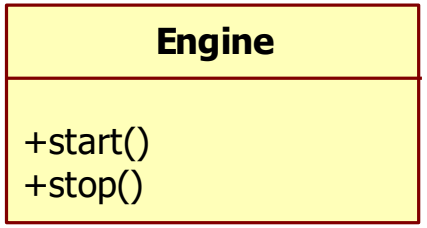
Coupling

- Controller vs engine
  - lowest
  - intermediate
  - highest

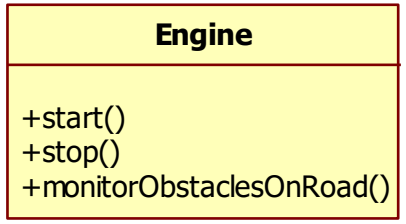


Cohesion

- Higher



- lower



Non functional properties

- Cost
- Schedule
- Staff skills

Properties – what to do

- Performance
  - Localise critical operations and minimise communications. Use large rather than fine-grain components.
- Security
  - Use a layered architecture with critical components in the inner layers.
- Safety
  - Localise safety-critical features in a small number of components.
- Availability
  - Include redundant components and mechanisms for fault tolerance.
- Maintainability
  - Use fine-grain, replaceable components.

Properties

- Using large-grain components improves performance but reduces maintainability.
- Introducing redundant data improves availability but makes security more difficult.
- Localising safety-related features usually means more communication, so degraded performance.

Properties, trade offs

- Not all properties can be satisfied
- Design is also about deciding tradeoffs
  - Ex security (add layers) vs. speed (avoid layers)
  - Ex. changeability (add abstraction layer to insulate from hardware change) vs. speed (avoid layers)
- Possibly, trade offs are decided at requirement time
  - Ex: requirement: security prevails on speed

Notations for formalization of architecture

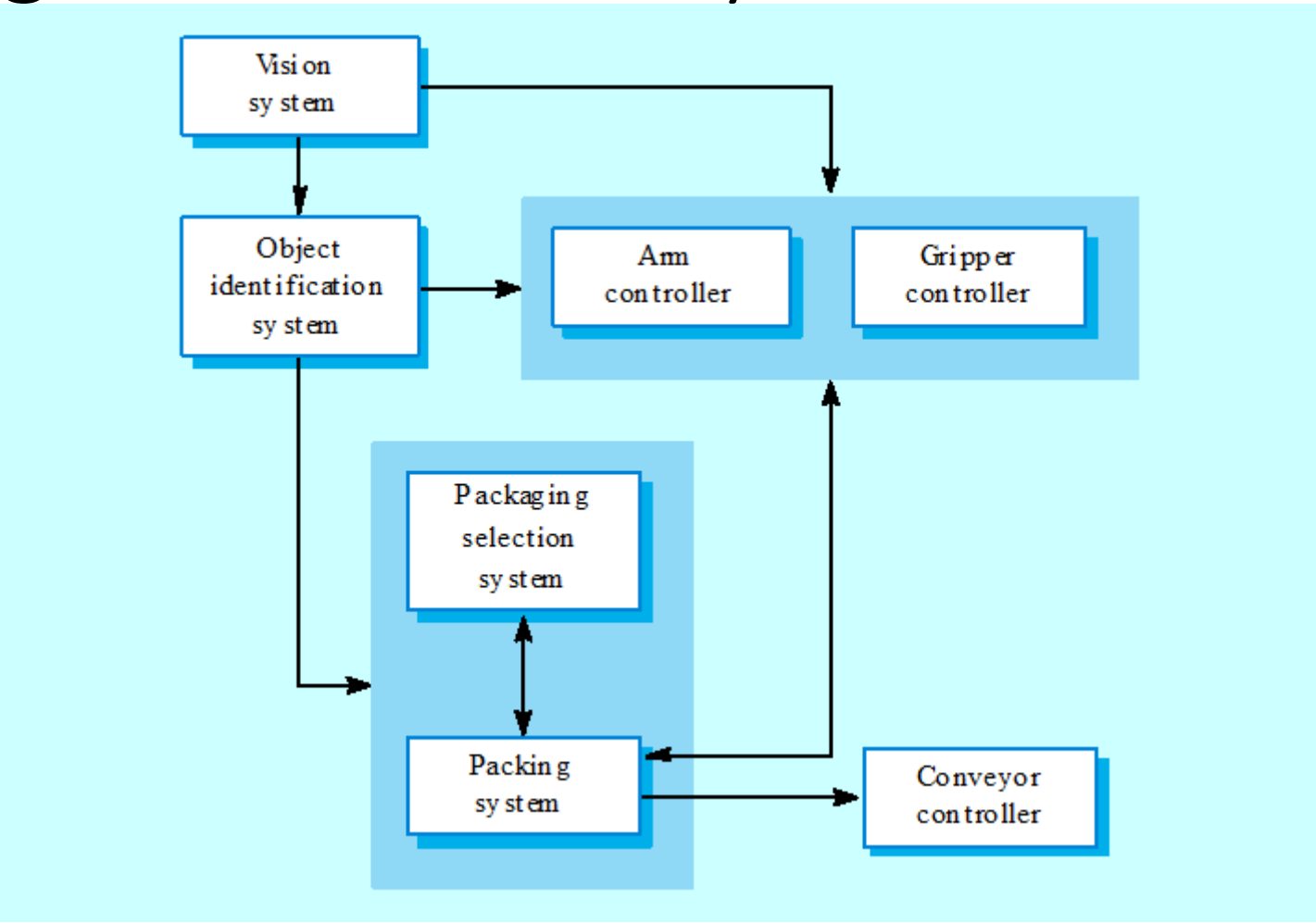
Formalizing the architecture

- Informal
  - box and lines
- Semiformal
  - UML diagrams
    - Structural views
      - Component, package diagrams
      - Class diagrams
      - Deployment diagram
    - Dynamic views
      - Sequence diagrams
      - State charts
- Formal ADL (Architecture description languages)
  - Many, ex C2 (component Connector)

Box and line diagrams

- Very abstract - they do not show the nature of component relationships nor the externally visible properties of the sub-systems.
- However, useful for communication with stakeholders and for project planning.

Packing robot control system



UML diagrams

- Structural view
  - Component or package diagram for high level view
  - Class diagram (inside each package or component)
  - Class description (for each class)

Heating control system

- Goal: control temp in house, using sensors in each room, and actuators (open close heating in each room)
- Choices – high level
  - One CPU, one process (no distribution, no concurrency)
  - Communication and control: procedure call
  - Layered style (at least partially)
- Choices – low level
  - Observer pattern

Heating control system

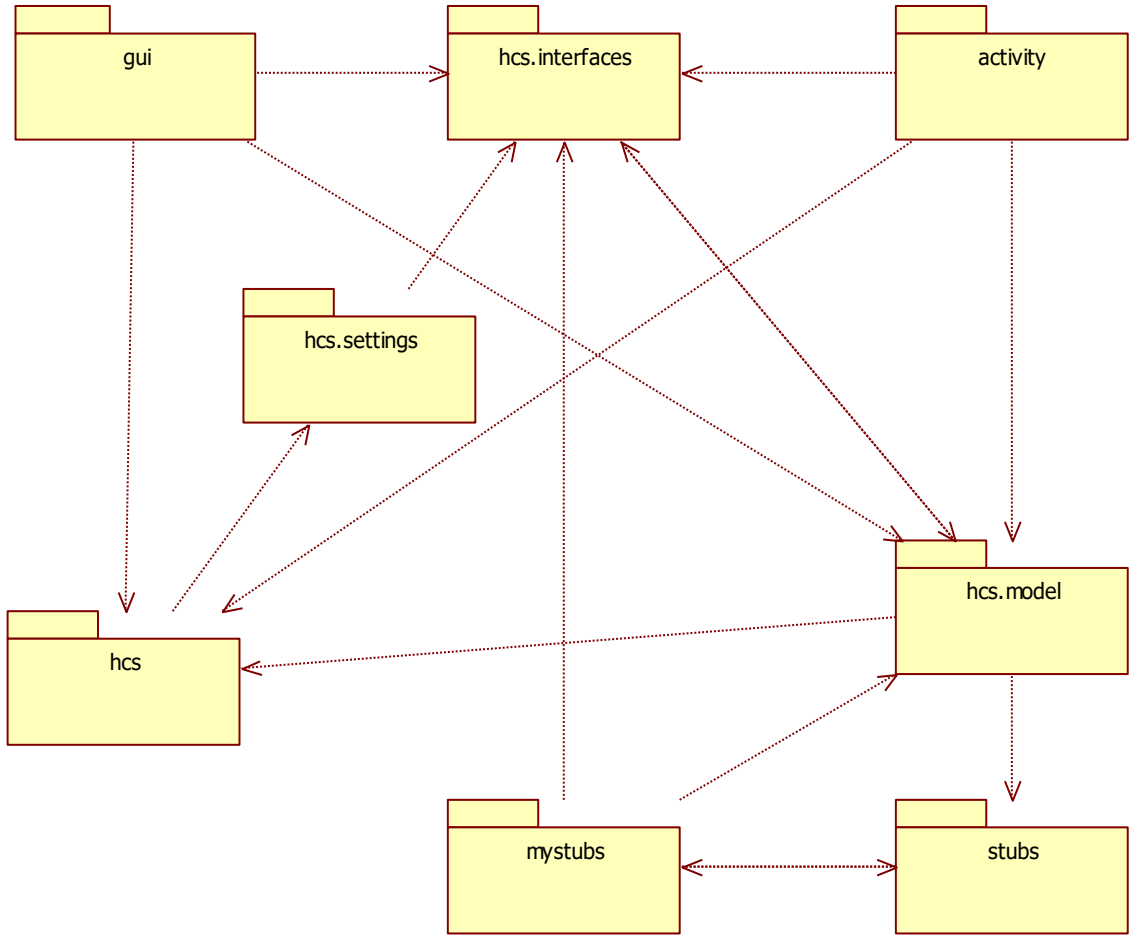
- Option 1
- High level:
  - One CPU, one process (no distribution, no concurrency)
  - Communication and control: procedure call
  - Layered style (at least partially)

Heating control system

- Option 2
- High level:
  - One CPU per room, one CPU per house (distribution, concurrency)
  - Communication and control: http calls (house controller calls rooms)

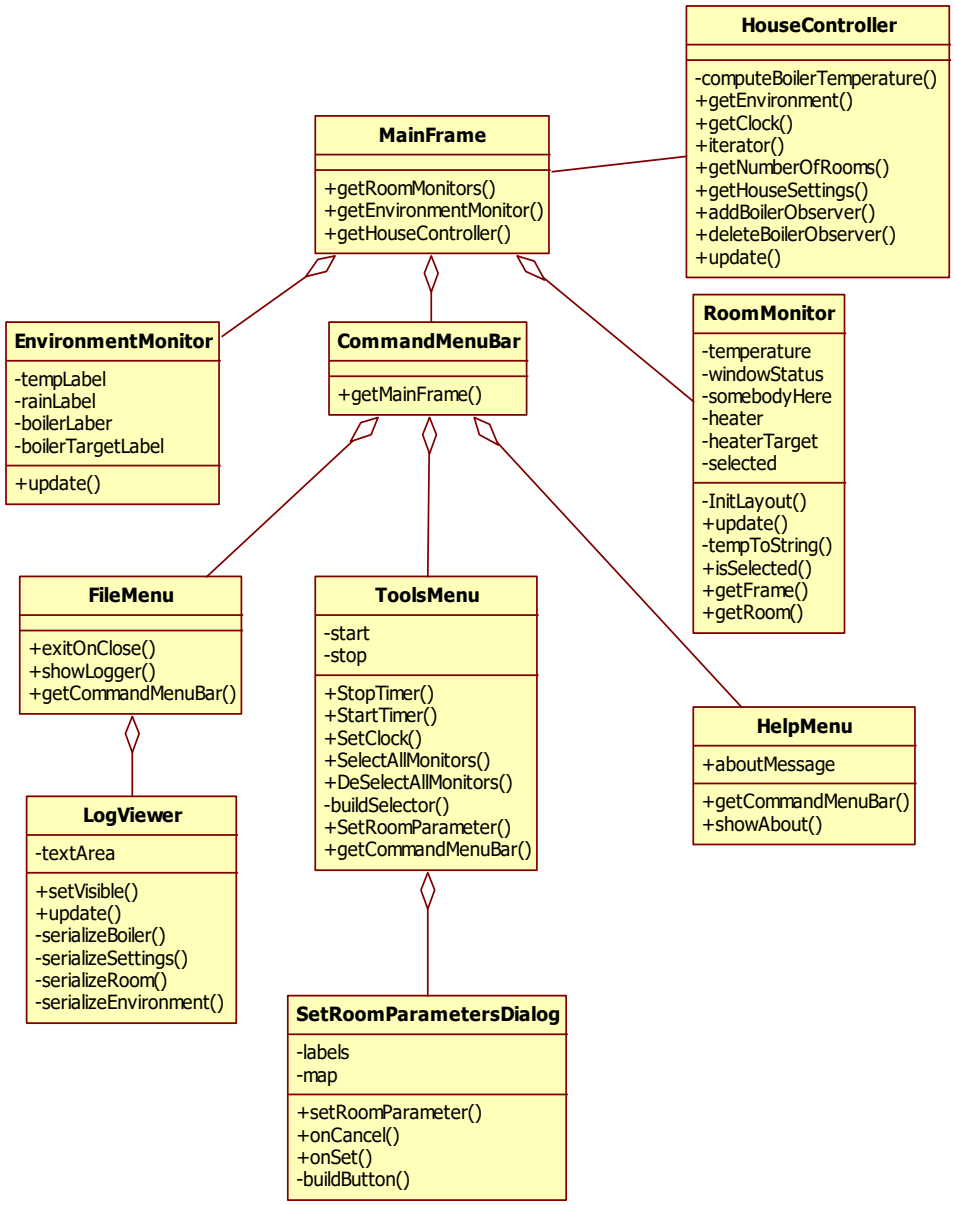
UML - structural

UML – package diagram



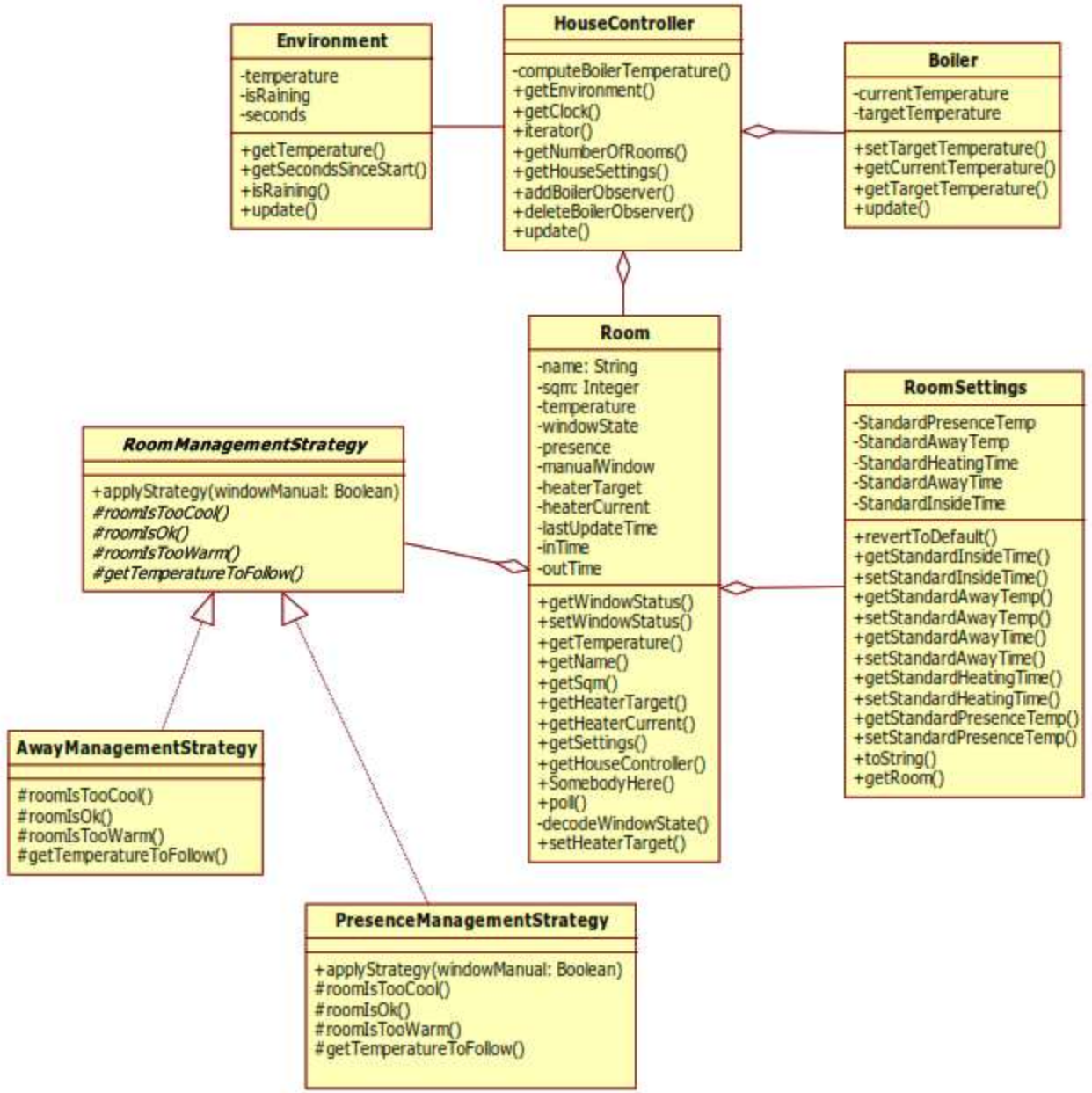
Design

- Package GUI



Design

- Package HCS



Class (HouseController)

- The main class in the heating control system integrates the logical model of the various parts of the house and performs high-level activities.
- computeBoilerTemperature()
  - Computes the desired water temperature in the boiler
- getEnvironment()
  - Navigates to the logical model of the environment
- getClock()
  - Navigates to the Clock
- iterator()
  - Returns an iterator to the contained Rooms
- getNumberOfRooms()
  - Returns the number of rooms
- getHouseSettings()
  - Navigates to the current global settings
- update()
  - Computes the next logical state of the system
- addBoilerObserver()
  - Adds an observer to the Boiler
- deleteBoilerObserver()
  - Removes an object from the list of Boiler observers

Structure and hierarchy

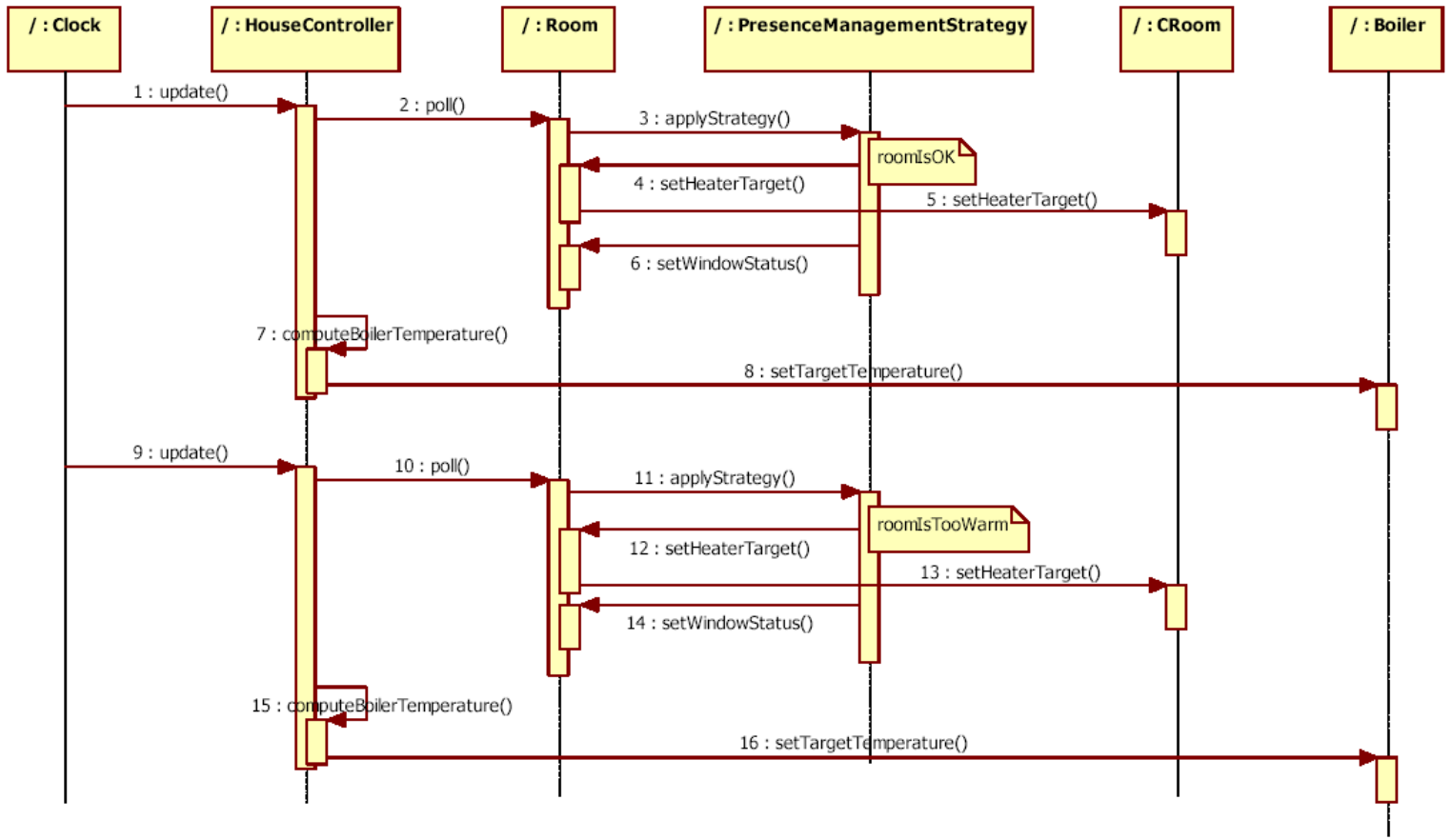
- UML helps in presenting structure in an organized (hierarchical) way
  - Packages in system
  - Classes in package
  - Attributes and methods in class
- Presentation is sequential, but the definition of such a structure requires several iterations

UML - dynamic

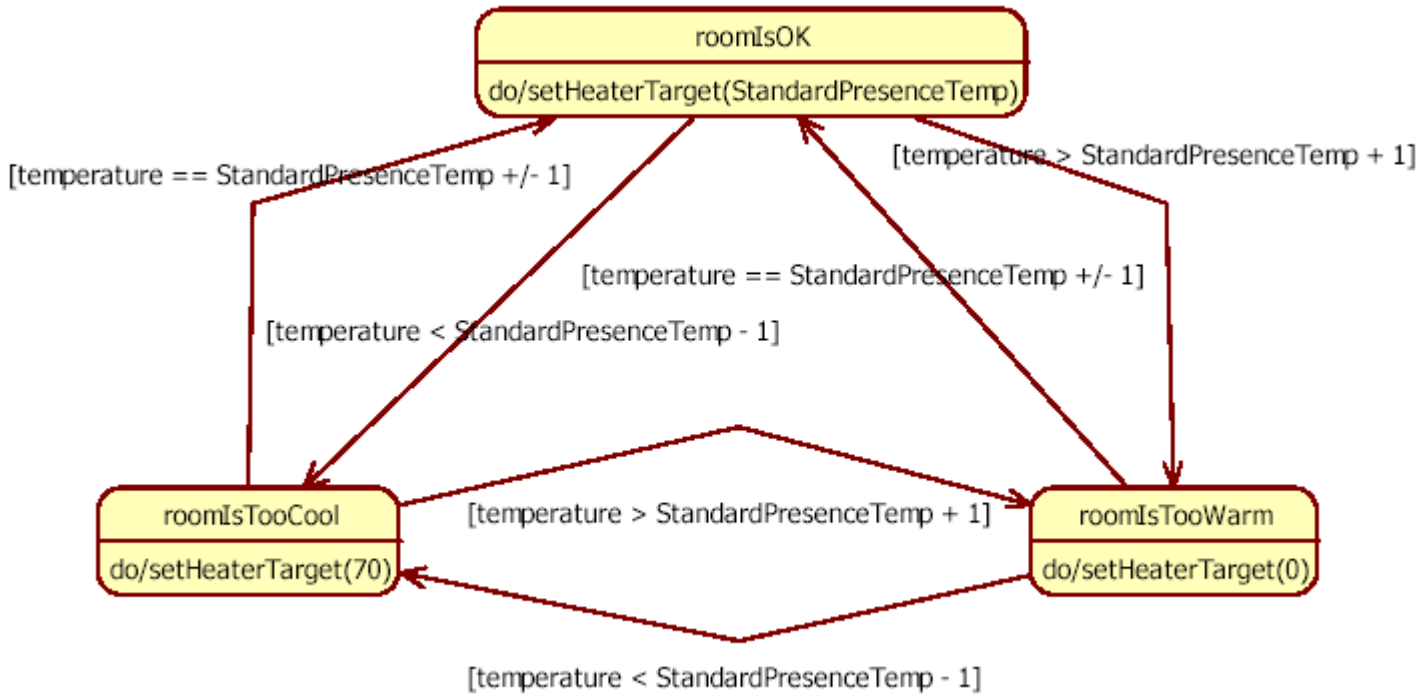
- State charts
- Sequence diagrams

Sequence

Sequence diagram for scenario 11:



State chart



Patterns

Patterns

- Reusable solutions
- To recurring problems
- In a defined context

- Cfr also dominant design in technology management area

Patterns

- Known, working ways of solving a problem

History

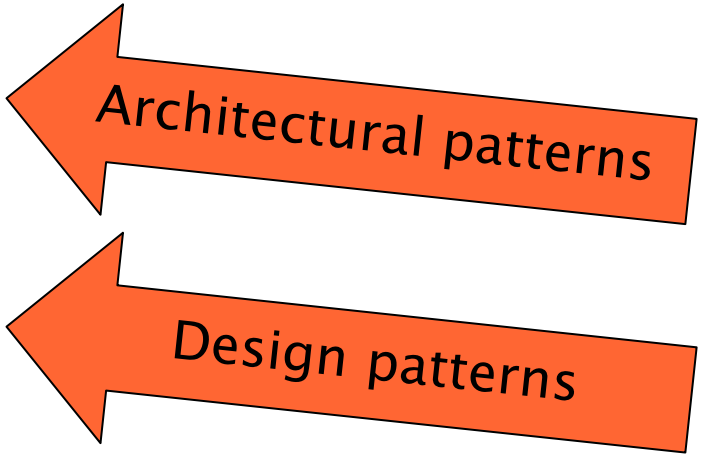
- Initially proposed by Christopher Alexander
- He described patterns for architecture (of buildings)
  - *The pattern is, in short, at the same time a thing, which happens in the world, and the rule which tells us how to create that thing and when we create it. It is both a process and a thing …*

Types of Pattern

- Architectural Patterns (or styles)
  - Address system wide structures
- Design Patterns
  - Leverage higher level mechanisms
- Idioms
  - Leverage language specific features

Patterns vs. process

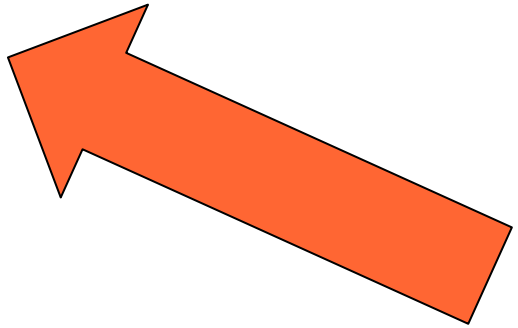
- 1 Architecture
- 2 Design
  - 2.1 High level
  - 2.2 Low level



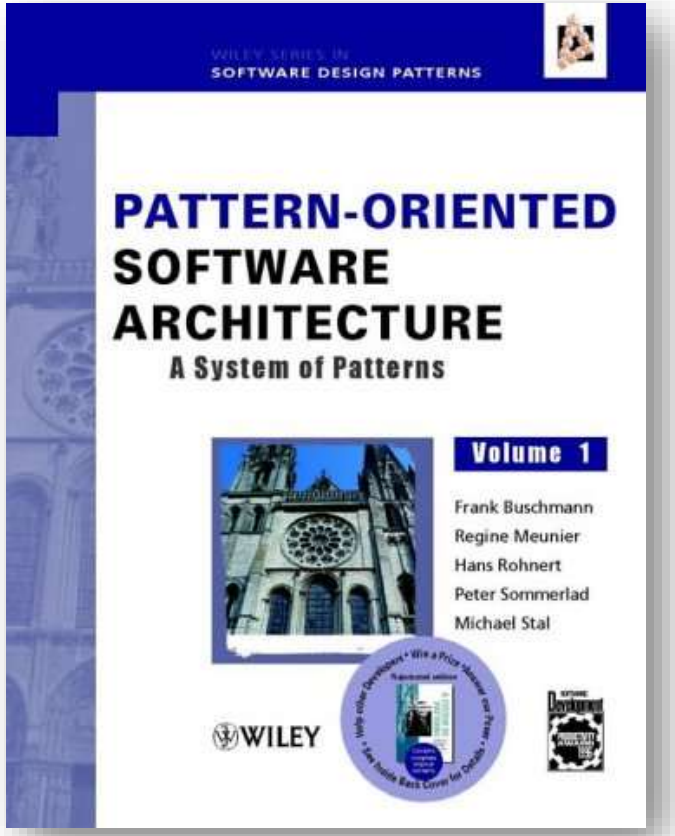
Architecture

Process

- 1 Architecture
- 2 Design
  - 2.1 High level
  - 2.2 Low level



Architectural patterns



Architectural patterns

- Repository
- Client server
- Layers
- Pipes and filters
- Broker
- MVC
- Microkernel
- Monolith vs Microservices

- A real system is usually influenced by many architectural patterns / styles

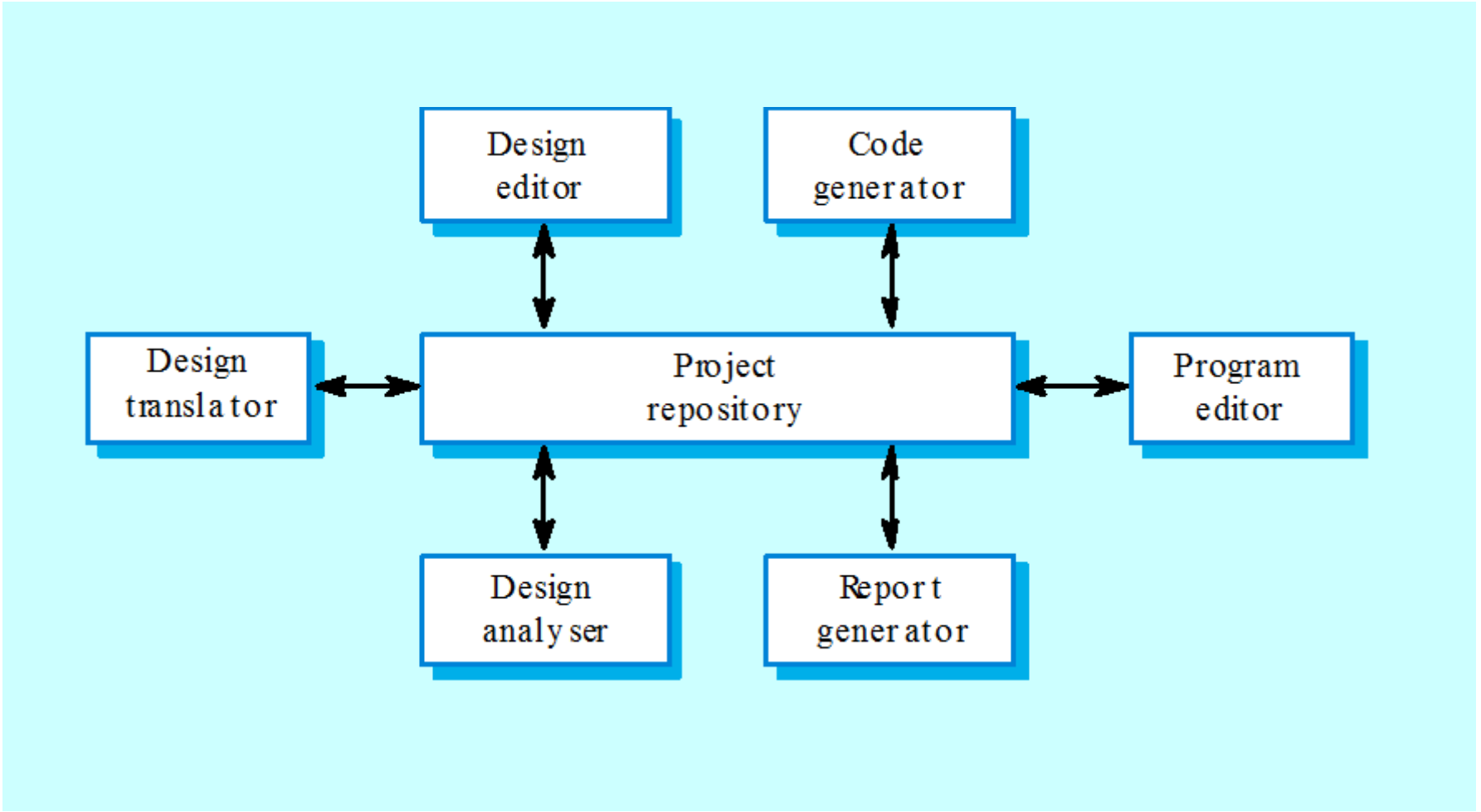
The repository style

- Sub-systems must exchange data. This may be done in two ways:
  - Shared data is held in a central database or repository and may be accessed by all sub-systems;
  - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used.

The repository style

- Subsystems exchange data only through the repository, by reading/writing files
  - No direct exchange through API
- The data model is the same for all subsystems
- The repository takes care (in the same way for all subsystems) for common services: backup, security, ..

IDE toolset architecture



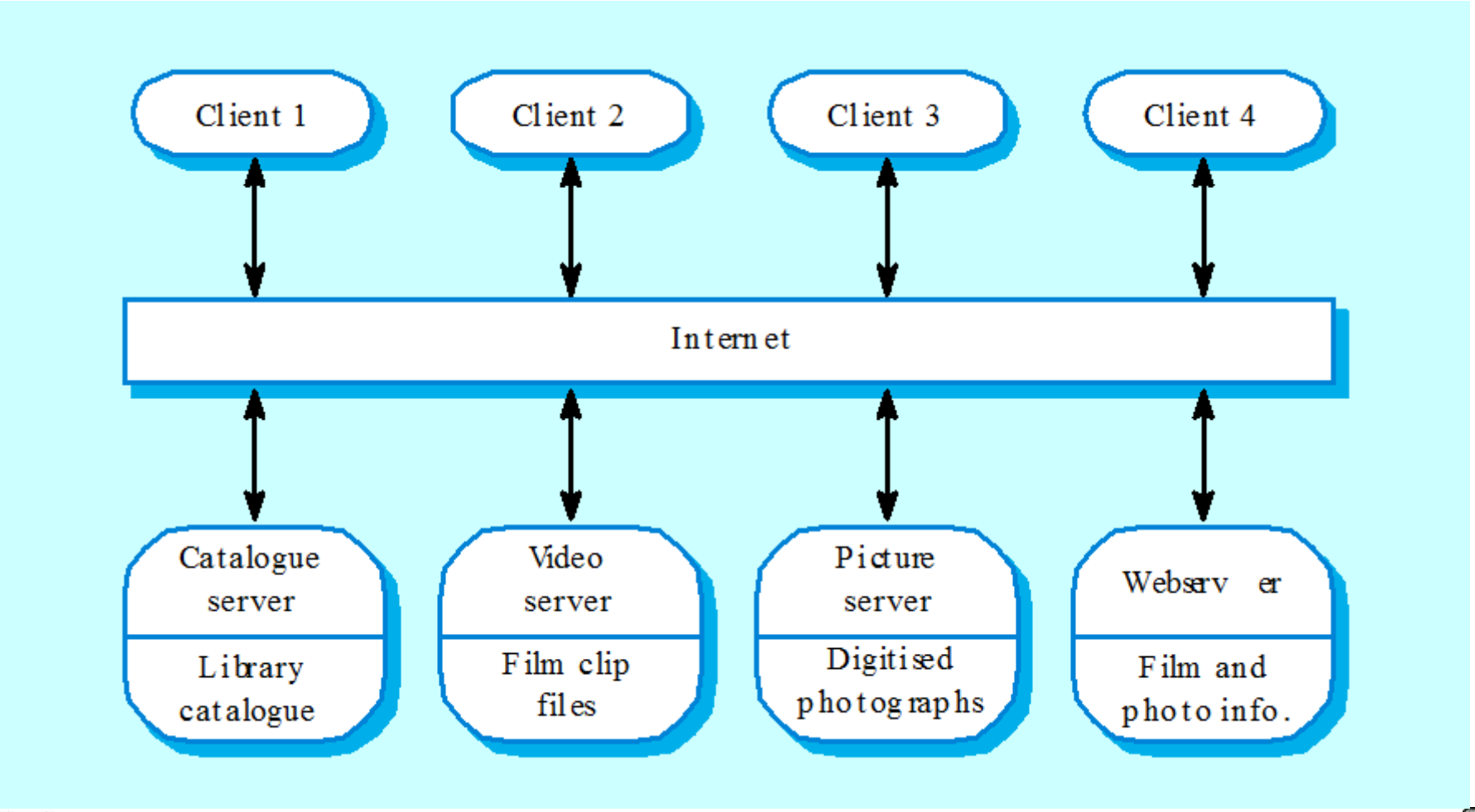
Repository style characteristics

- Advantages
  - Efficient way to share large amounts of data;
  - Sub-systems need not be concerned with how data is produced
  - Centralised management, e.g. backup, security
  - The sharing model is published as the repository schema.
- Disadvantages
  - Sub-systems must agree on a repository data model. Inevitably, a compromise;
  - Data evolution is difficult and expensive;
  - No scope for specific management policies;
  - Difficult to distribute efficiently.

Client-server Model

- Distributed system model which shows how data and processing is distributed across a range of components.
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services.
- Network which allows clients to access servers.

Film and picture library



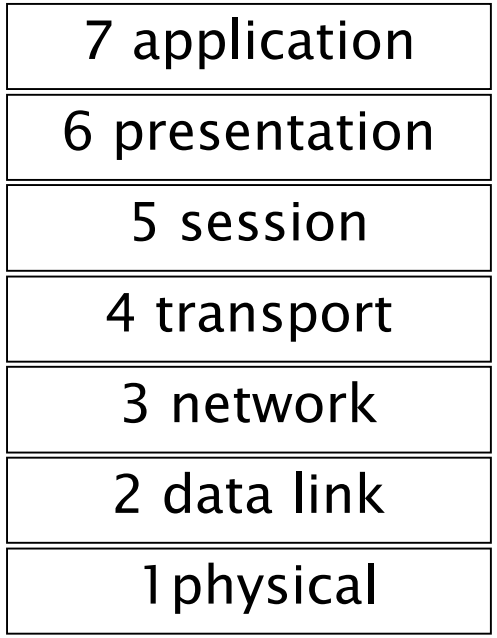
Client-server characteristics

- Advantages
  - The distribution of data is straightforward;
  - Makes effective use of networked systems. It may require cheaper hardware;
  - Easy to add new servers or upgrade existing servers.
- Disadvantages
  - There is no shared data model, so sub-systems use different data organisations. Data interchange may be inefficient;
  - Redundant management in each server;
  - No central register of names and services - it may be hard to find out what servers and services are available.

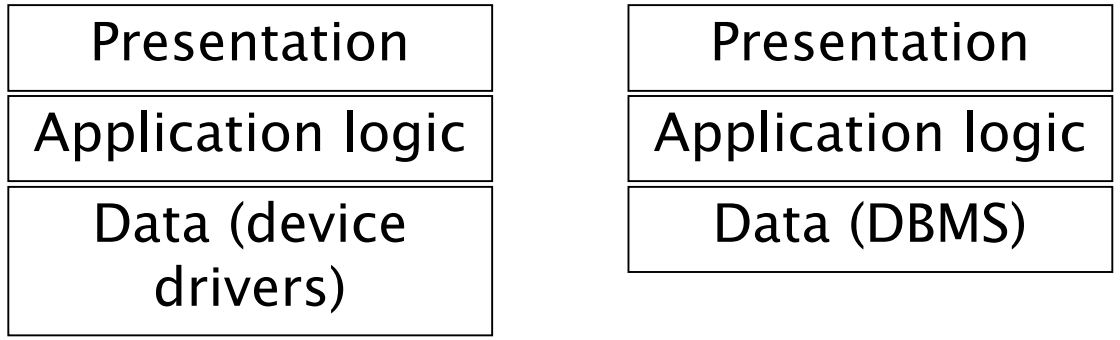
Abstract machine (layered) model

- Used to model the interfacing of sub-systems.
- Organises the system into a set of layers (or abstract machines) each of which provide a set of services.
- Constraint: layer uses only services from adjacent layer
- Advantages
  - In design: each layer is about a problem (separation of concerns)
  - In evolution: when a layer interface changes, only the adjacent layer is affected.
- Problems
  - Sometimes artificial to structure systems in this way.

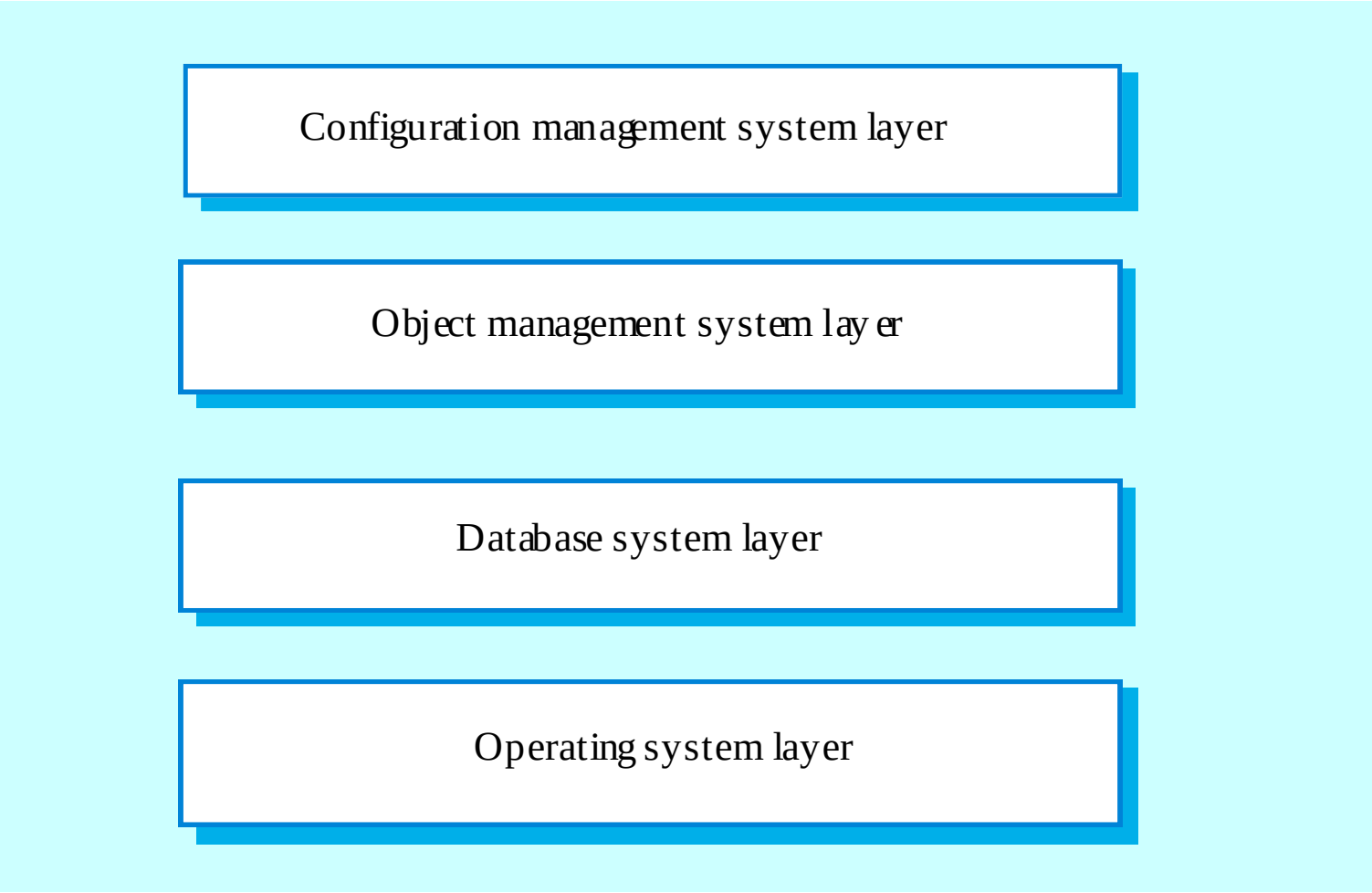
ISO Osi model



3 tier architecture



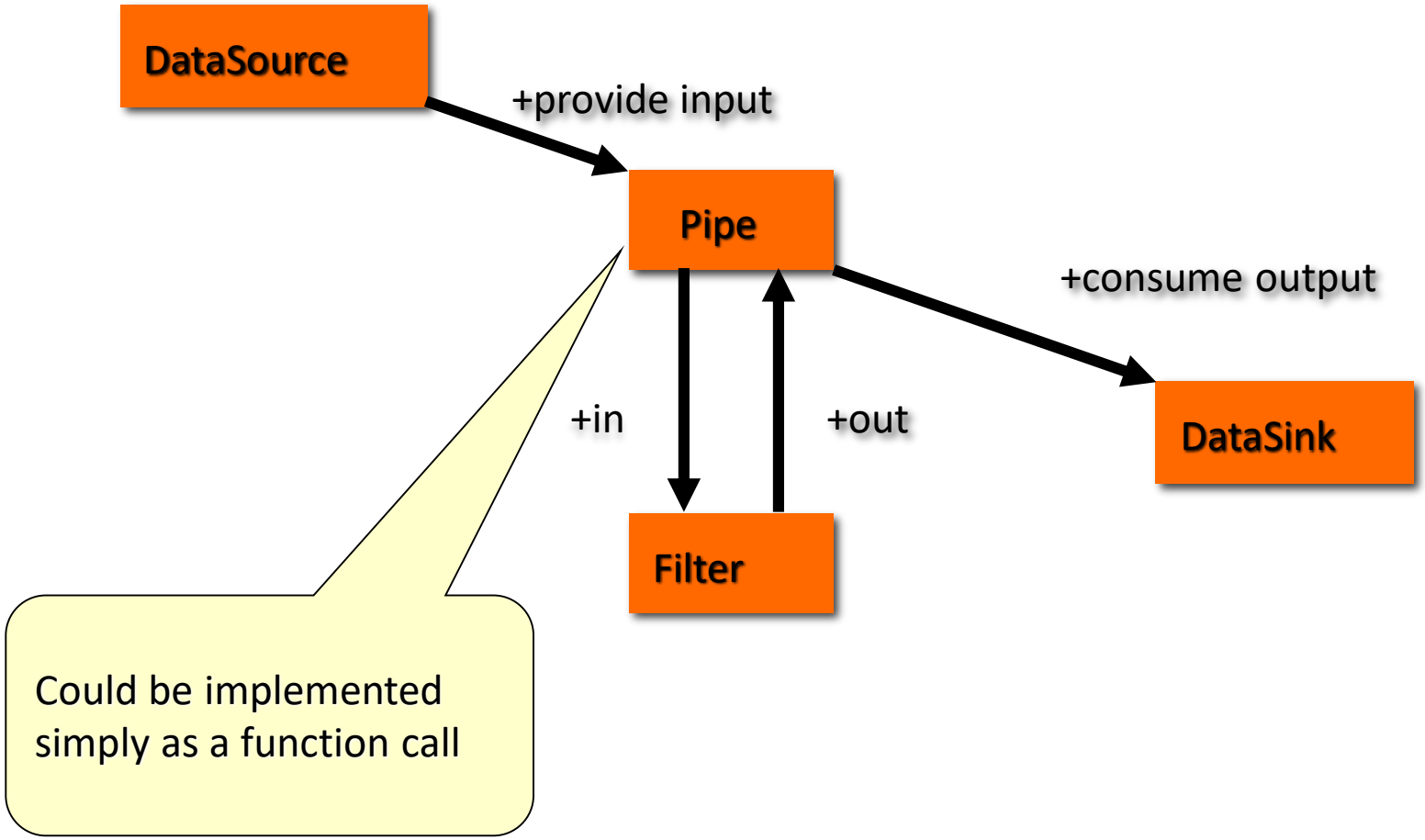
Version management system



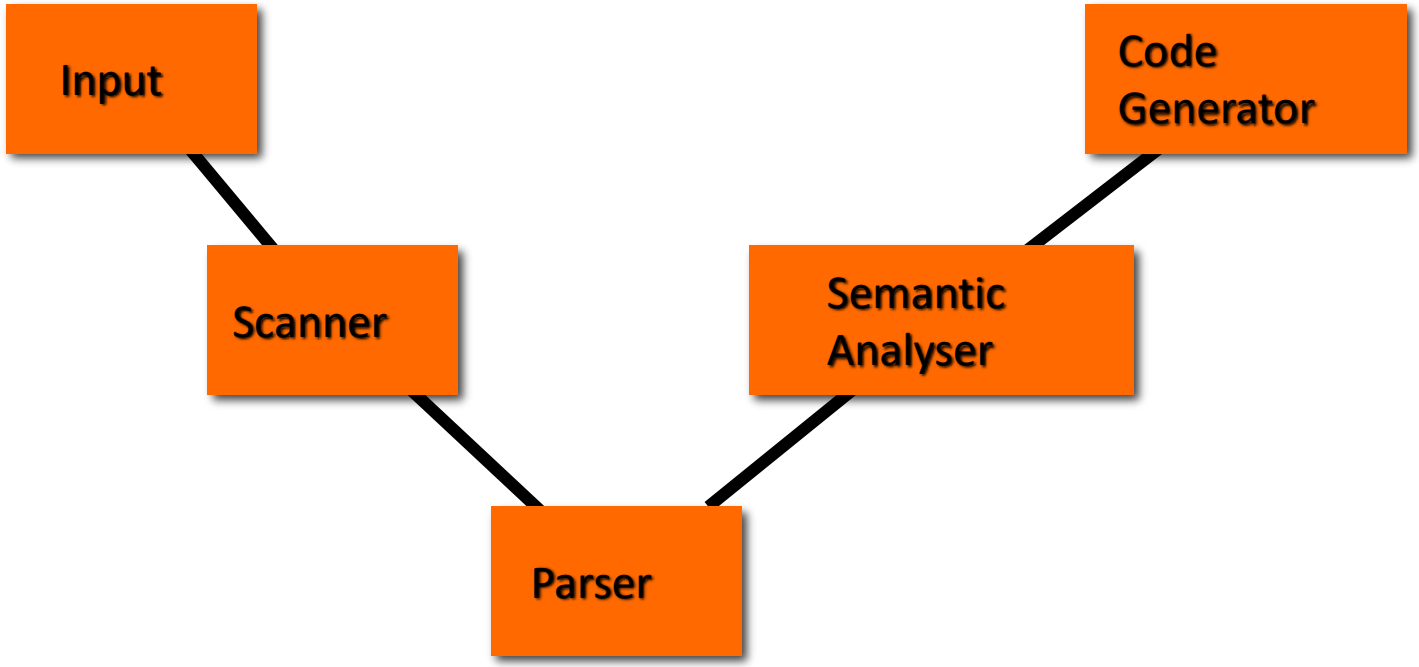
Pipes & Filters

- Context
  - We need to process data streams according to several steps
- Problem
  - Must be possible recombining steps
  - Non-adjacent steps do not share info
  - The user storing data after each step may result into errors and garbage

Pipes & Filters

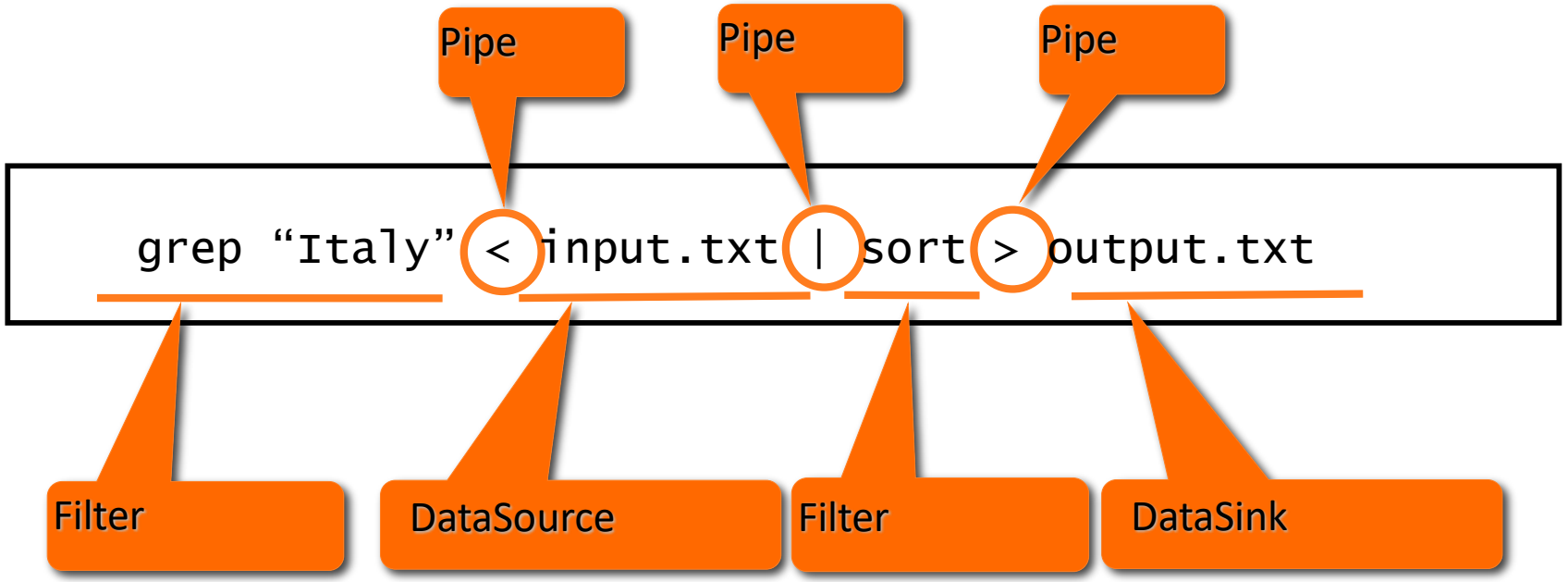


Pipes & Filters Example

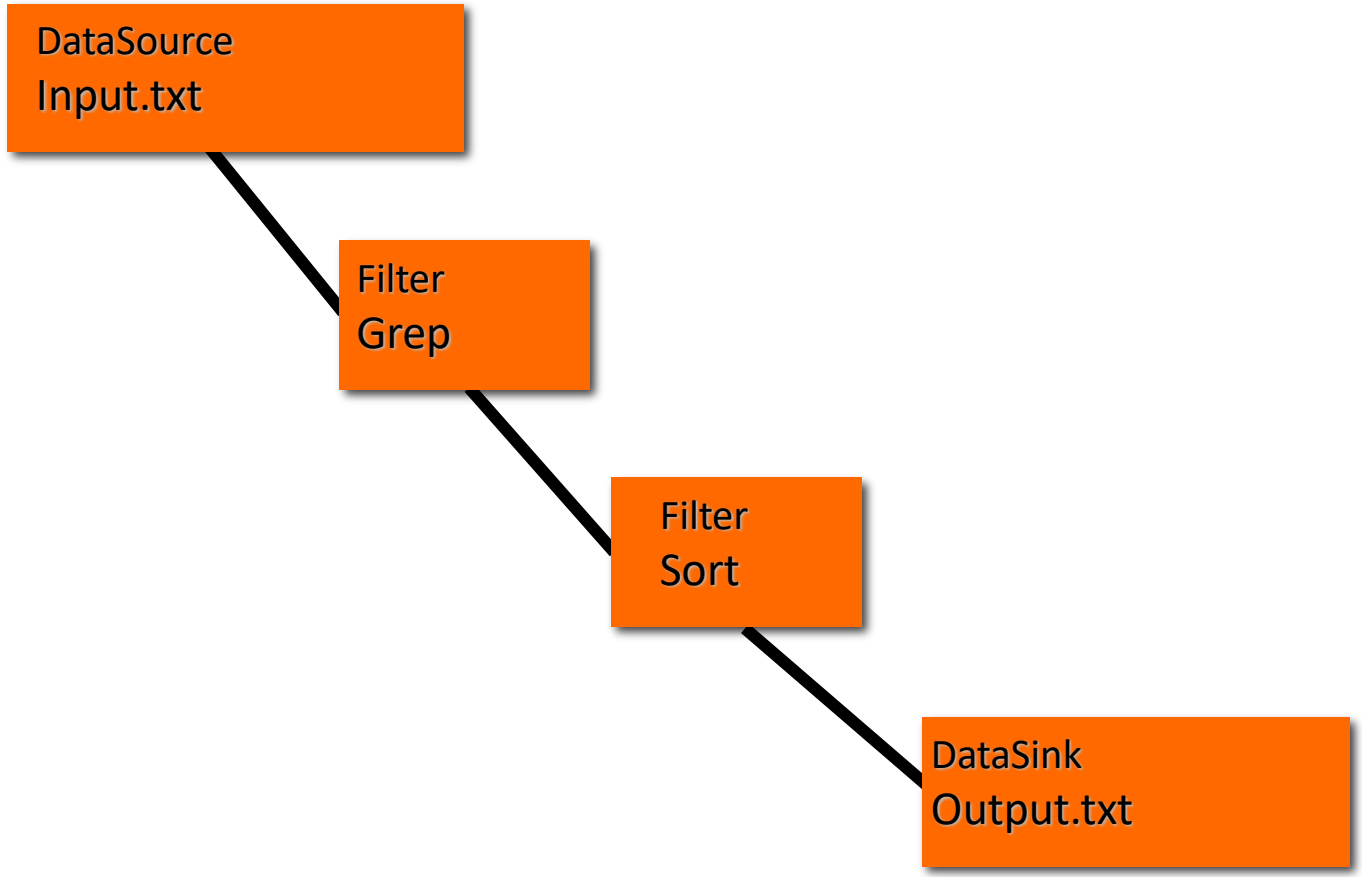


Pipes & Filter Example

Unix shell commands



Pipes & Filter Example



Pipes & Filter Example

Input.txt

```
Rome, Italy
Milan, Italy
Turin, Italy
Paris, France
Marseille, France
Brussels, Belgium
Munich, Germany
Berlin, Germany
```

Pipes & Filter Example

grep “Italy” < Input.txt

```
Rome, Italy
Milan, Italy
Turin, Italy
Paris, France
Marseille, France
Brussels, Belgium
Munich, Germany
Berlin, Germany
```

Pipes & Filter Example

| sort > output.txt

```
Rome, Italy
Milan, Italy
Turin, Italy
```

Pipes & Filter Example

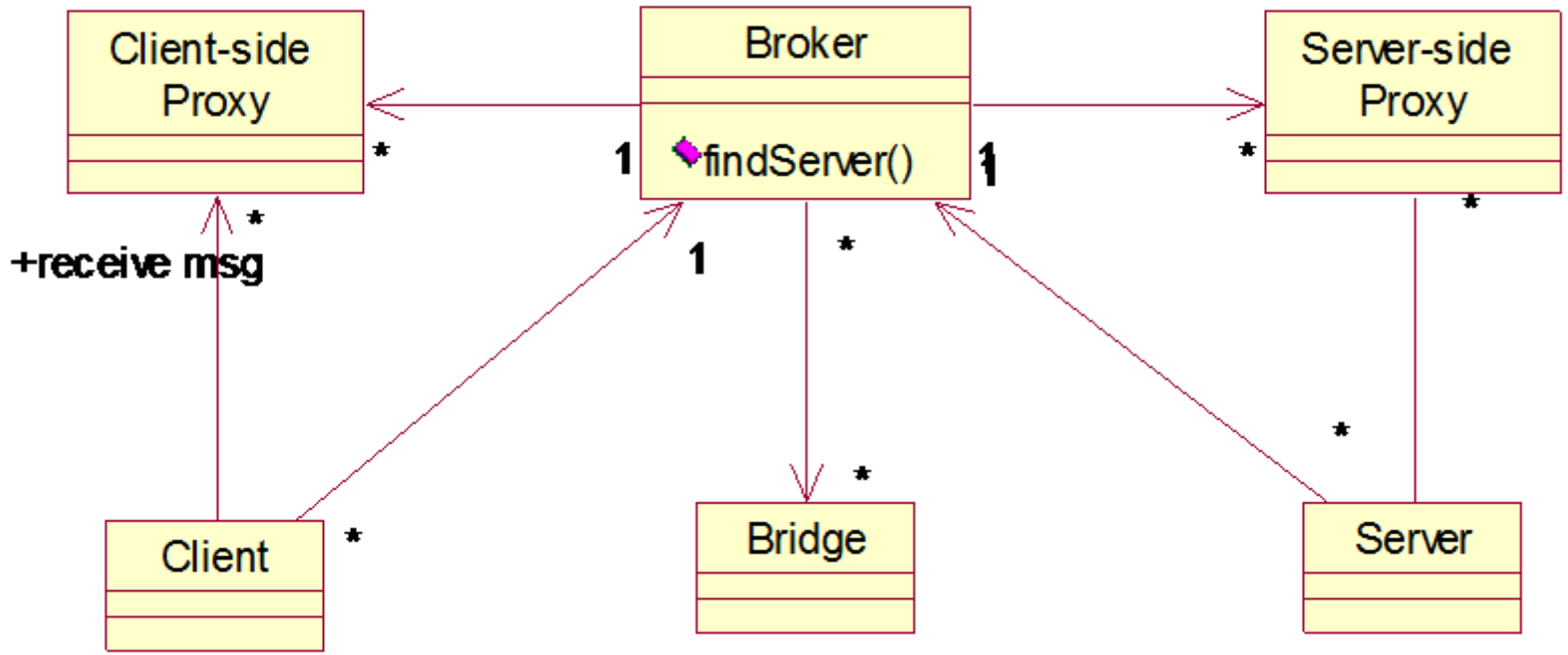
output.txt

```
Milan, Italy
Rome, Italy
Turin, Italy
```

Broker

- Context
  - Environment with distributed and possibly heterogeneous components
- Problem
  - Components should be able to access others
    - Remotely
    - Location independently
  - Components can be changed at run-time
  - Users should not see too many details

Broker



Scenario

- Servers register their services to broker
- Client requests service to broker in a defined (by broker) format
- Broker finds servers offering that service, selects one
- Broker forwards request to server (possibly changing format), obtains response, forwards request to client

- Advantages
  - Decoupling (see coupling concept): client does not need to know
    - which are the servers
    - what protocol they use

Ex: insurance broker

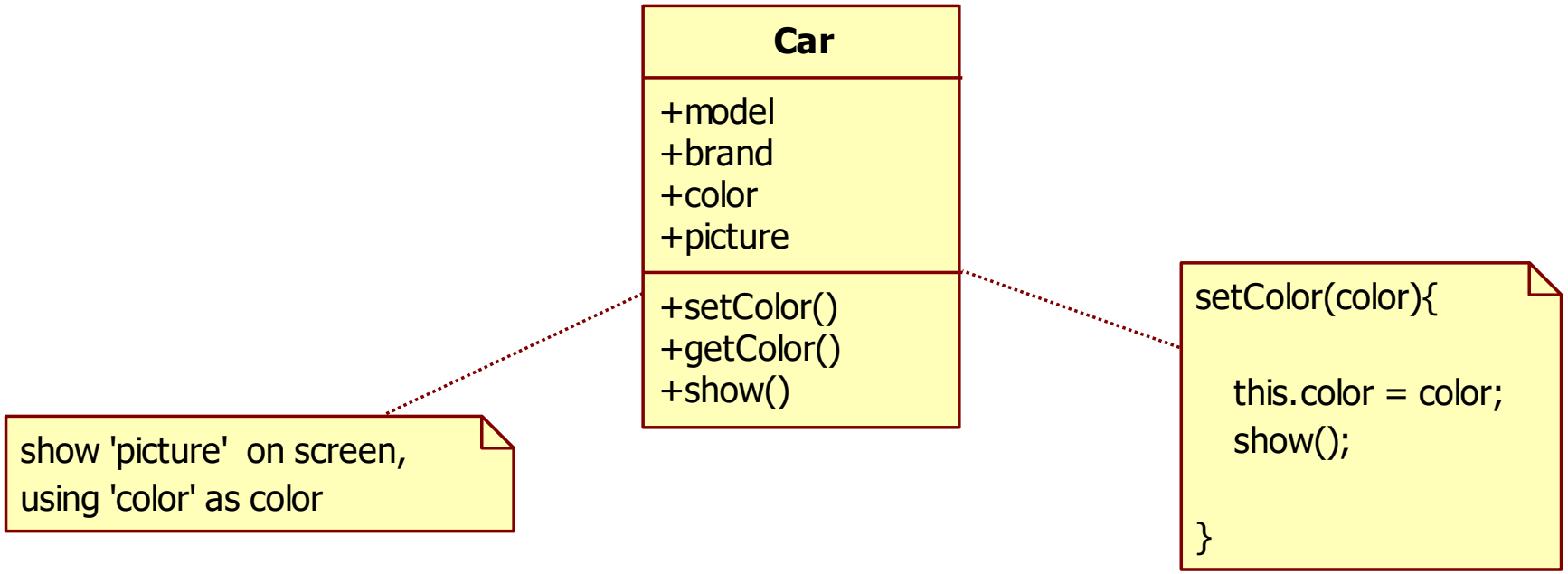
- Each online insurance company has a web site, customer can get offer for car insurance
  - Each web site has a different GUI and different steps to require information and finally produce offer

- Broker (or offer comparator): 6sicuro.it segugio.it facile.it
  - One single entry point to get customer information
    - Send offer request to several online insurances (data and protocol translation)
    - Receive offers (data translation)
    - Send back offers to customer

MVC - Problem

- Show data to user, manage changes to data
  - Option1: one class
  - Option2: MVC pattern

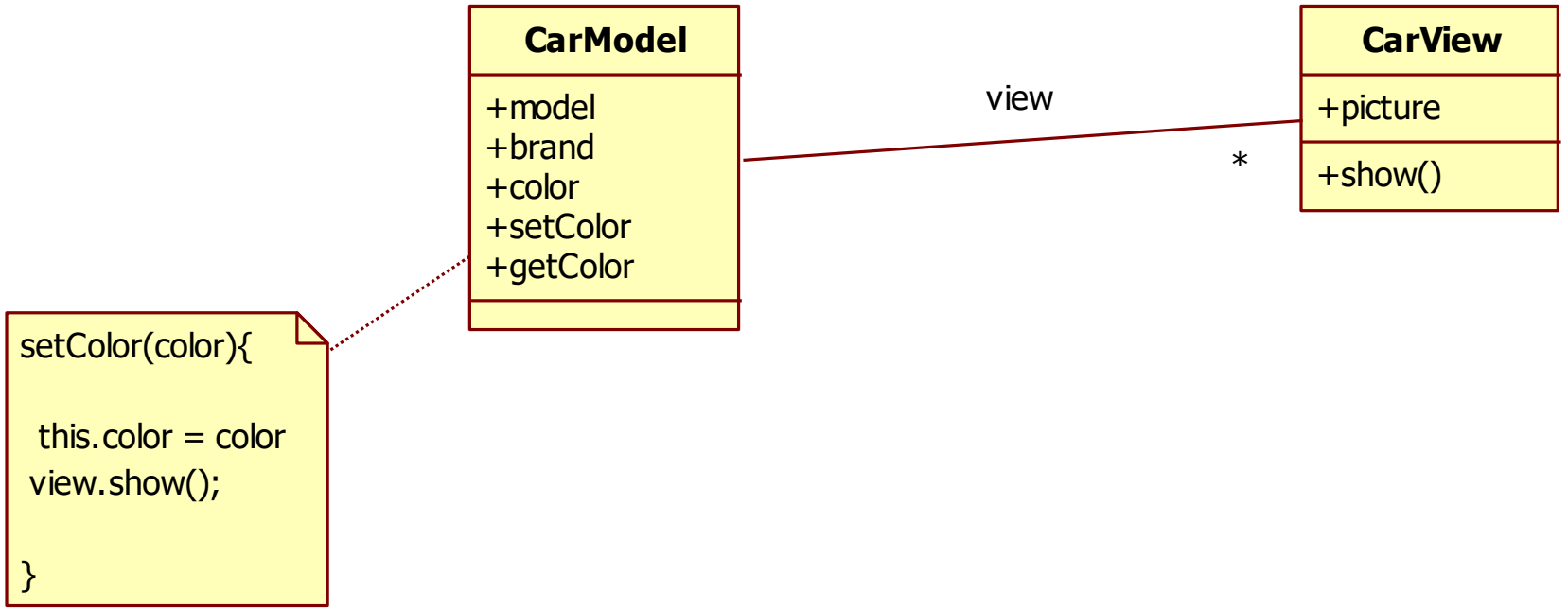
Option1



Option1

- Pro
  - Easy
- Con
  - What if two (three..) pictures?

Option2



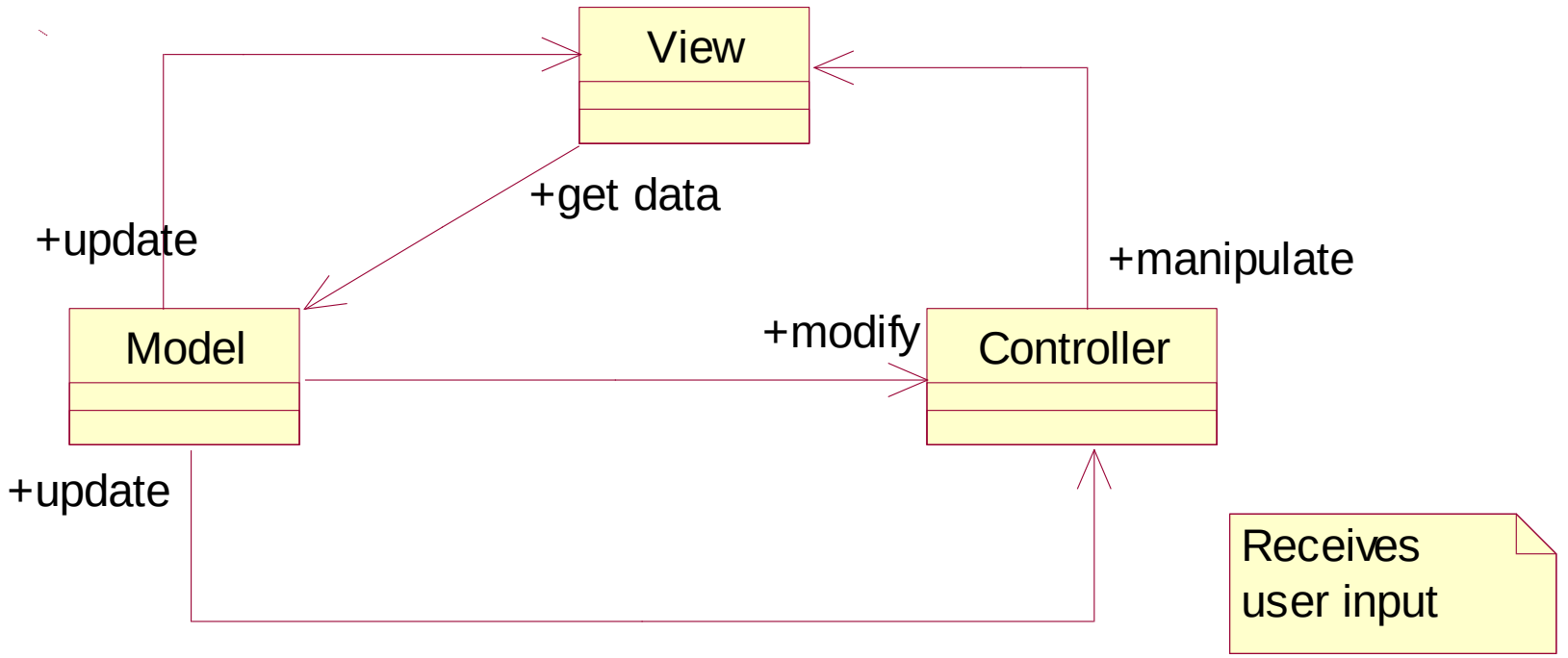
MVC

- Context
  - Interactive applications with flexible HCI
- Problem
  - The same information is presented in different ways/windows
  - Windows must present consistent data
  - Data changes
- Goal (product property)
  - Maintainability, portability

MVC

- Model
  - Responsible to manage state (interfaces with DB or file system)
- View
  - Responsible to render on UI
- Controller
  - Responsible to handle events from UI

MVC



MVC

- Pros
  - Separation of responsibilities
    - Many different views possible
    - Model and view can evolve independently (maintainability)
- Cons
  - More complexity (less performance)

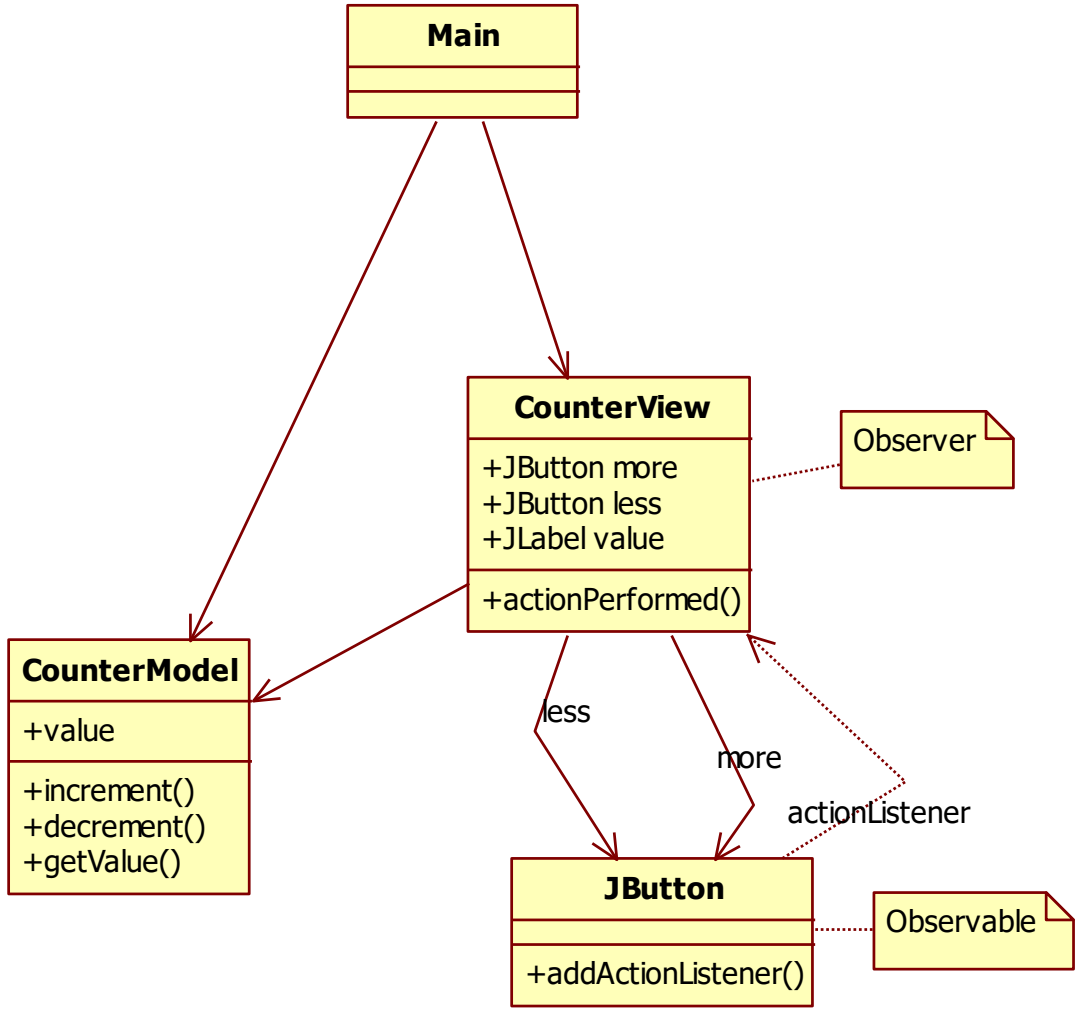
Execution flow

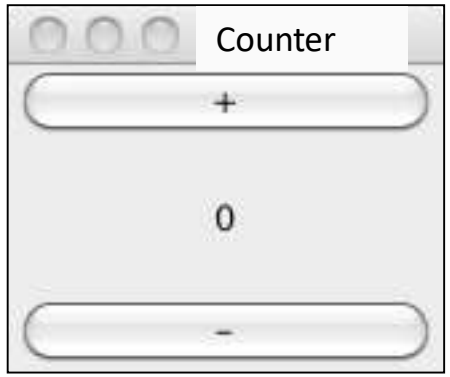
- There is no predefined order of execution
  - Operations are performed in response to external events (e.g. mouse click)
  - Event handling is serialized
  - To execute operations in parallel, threads must be used
  - Method main in GUIs has the only goal of instantiating the graphical elements

MVC implementations

- Given the high-level idea
- Different implementations happen in different environments
  - Java
  - C#
  - Android
  - IoS

MVC in Java (MV)





Code Example

```
class CounterView implements ActionListener {
  private CounterModel model;
  private JLabel valueLabel;
  private JButton more;
  private JButton less;
```

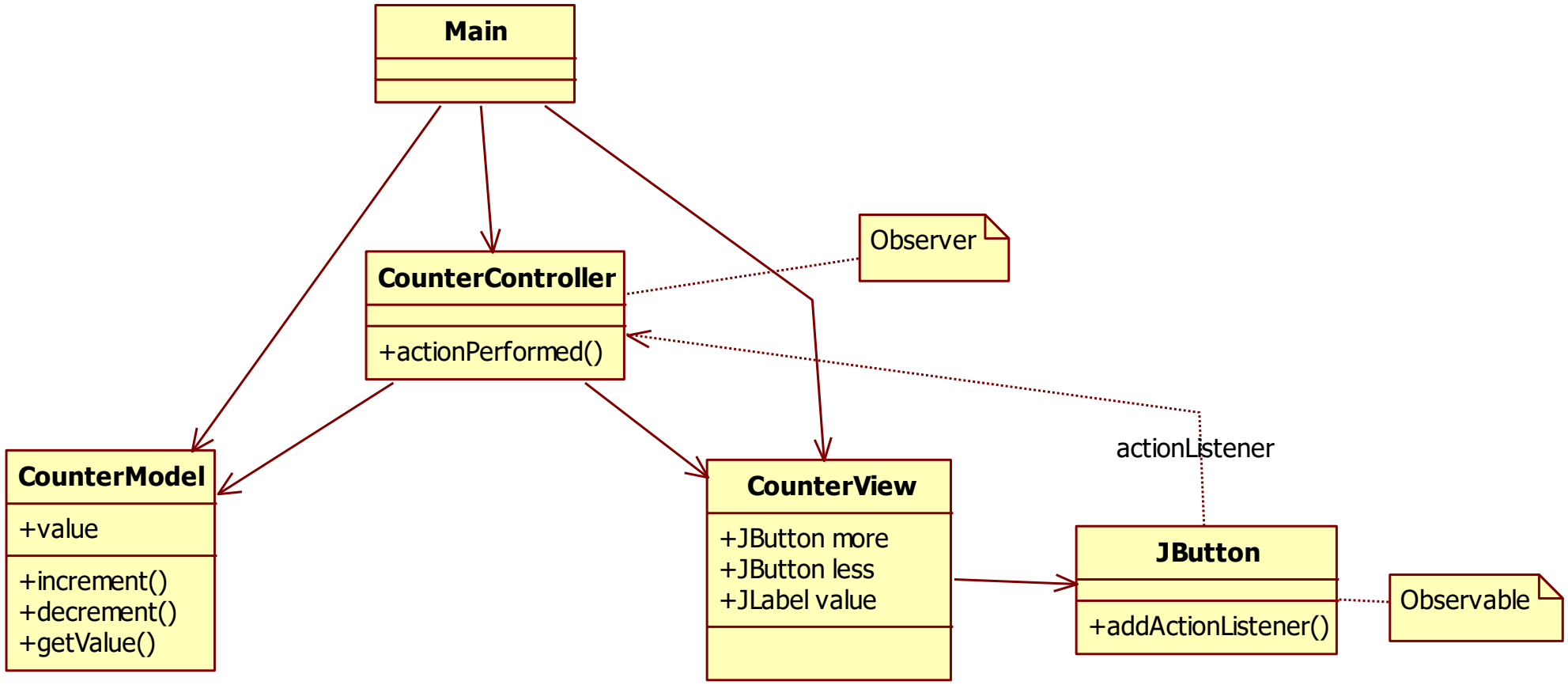
```
public CounterView(CounterModel m, JPanel panel) {
  model = m;
  int value = model.getValue();
  panel.add(new JLabel("counter"));
  panel.add(valueLabel= new JLabel(Integer.toString(value)));
  more = new JButton("more");
  less = new JButton("less");
  panel.add(more);
  panel.add(less);
  more.addActionListener(this);
  less.addActionListener(this);
}
public void update(){
  valueLabel.setText(Integer.toString(model.getValue()));
}

public void actionPerformed(ActionEvent arg0) {
  Object o = arg0.getSource();
  if (o== more) model.increment();
  if (o == less) model.decrement();
  update();
}
```

Code Example

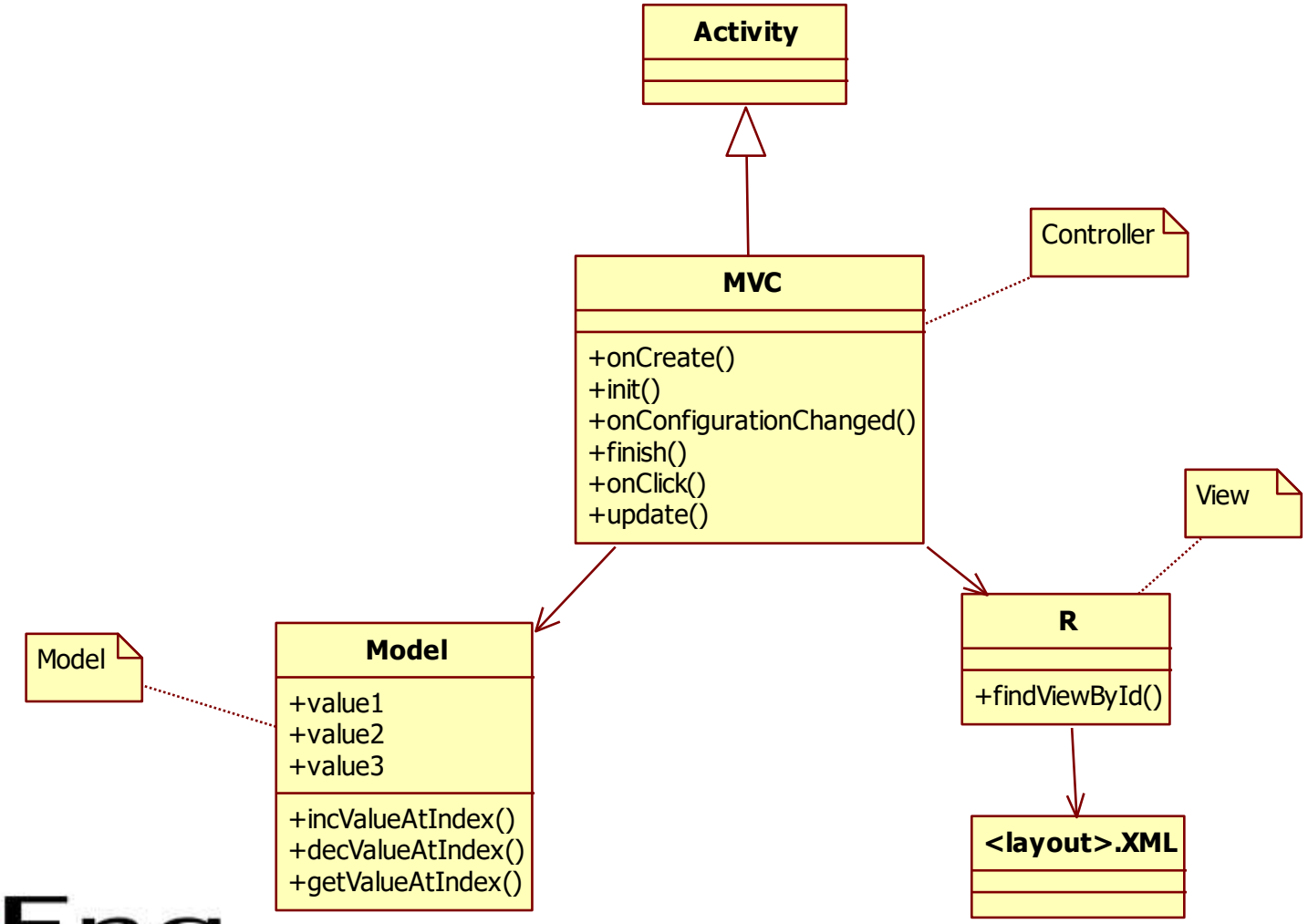
```
public class CounterModel {
  private int value;
  public void increment(){ value++;}
  public void decrement(){ value--;}
  public int getValue(){ return value;}
}
```

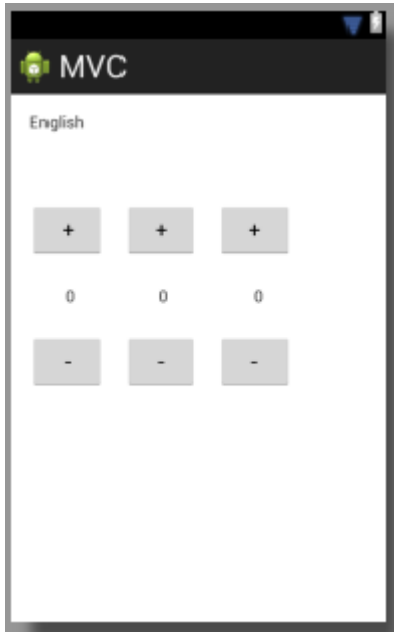
```
public class MainMV {

public static void main(String[] args) {

  JFrame frame = new JFrame();
  JPanel panel = new JPanel();
  panel.add(new JLabel("here"));
  frame.setContentPane(panel);
  frame.setSize(300,100);
  frame.setVisible(true);
  frame.repaint();

  CounterModel m = new CounterModel();
  CounterView v = new CounterView(m, panel);
}
```

MVC in Java (MVC)

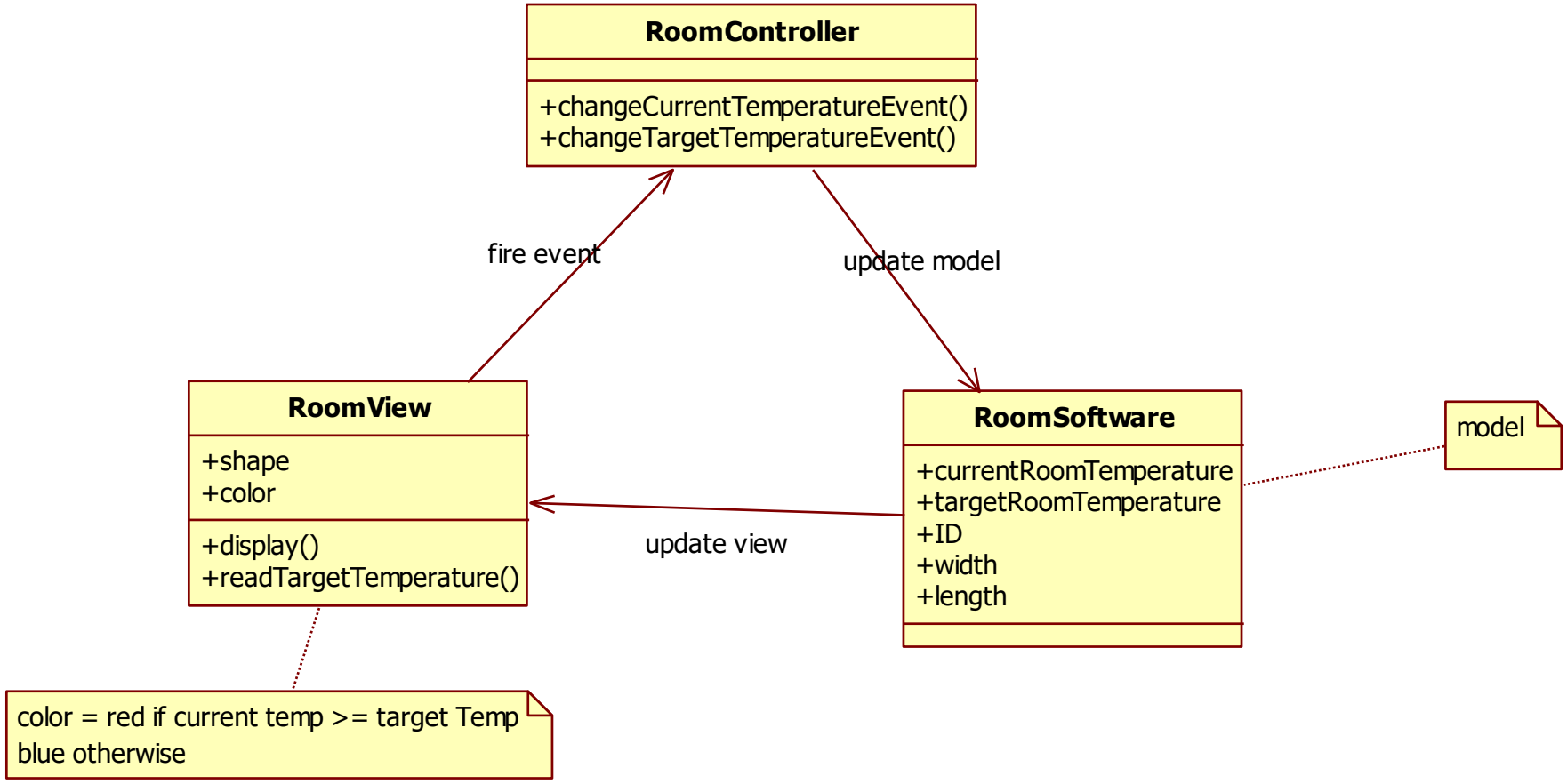


MVC in Android





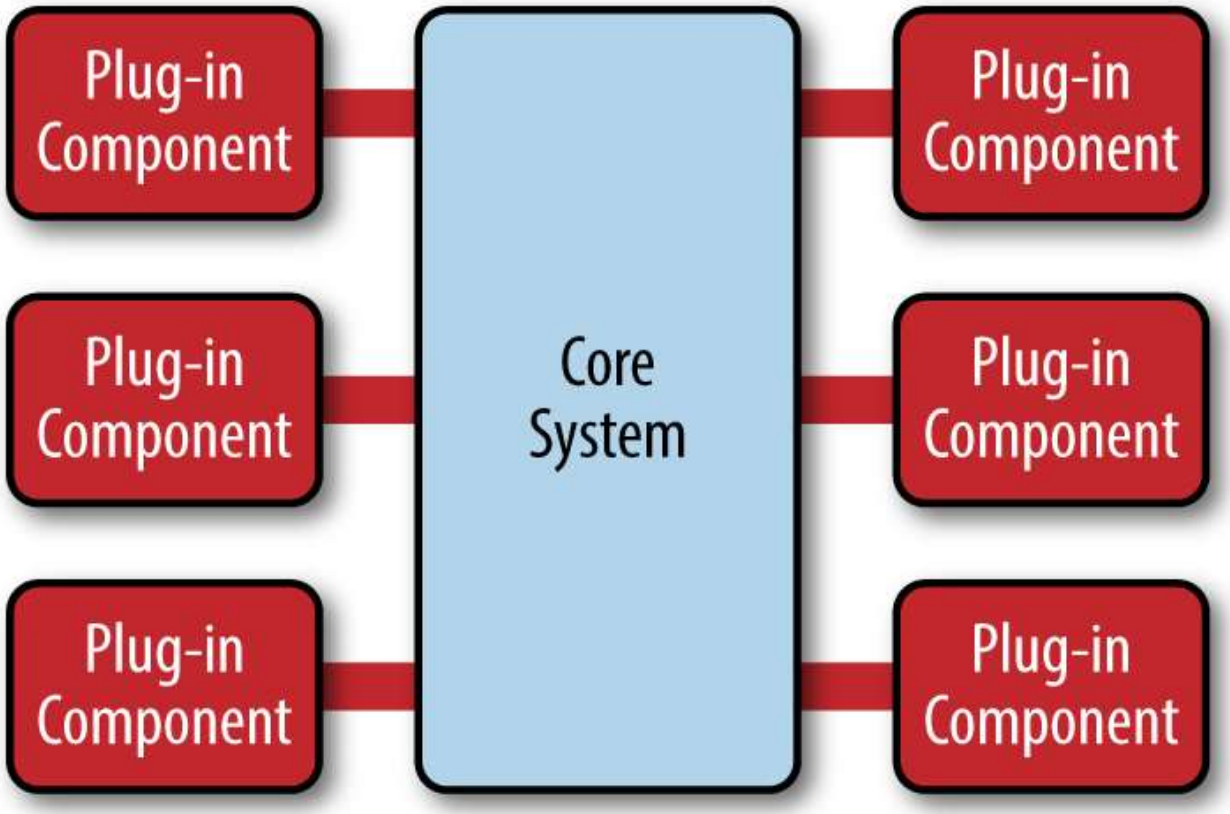
In heating control system



Microkernel

- Context
  - Several APIs (aka services) insist on a common core
- Problem
  - HW and SW evolve continuously and independently
  - The platform should be:
    - Portable
    - Extendable
- Solution
  - Isolate a stable core of services
  - Extend core with plugins (handle evolution and variability of other services)

Microkernel



Microkernel

- Example: web browser

- Core: capability of interacting with server
  - Handle http(s) communication
  - Render html
- Plugins:
  - Music player
  - Pdf viewer
  - Word/Excel viewer
  - …

Monolith and Microservices

Two different approaches in developing a complex application

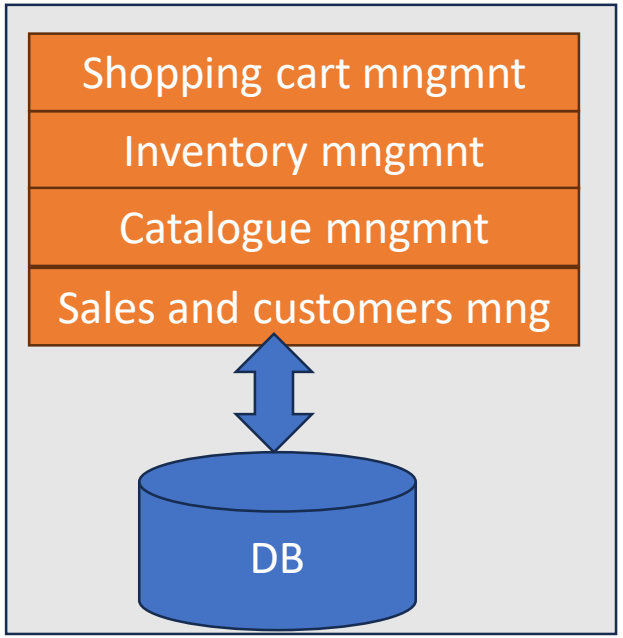
Monolith

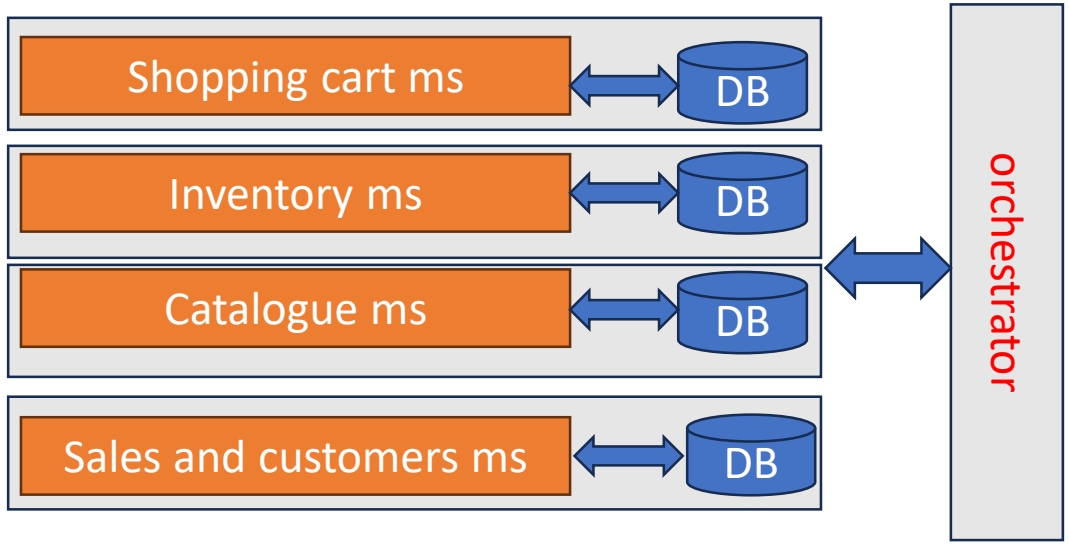
- One single db
- All services insisting on it
- Direct calls between services or data shared via db

Microservices

- Several microservices
- Each implements a service, with own db
- Communication via REST APIs

Ex e-commerce application





Microservices

- Microservice = one executable running in its process (real or virtual machine)
- Microservices communicate via http calls, (RESTFul APIs)
- Application made of many communicating microservices
  - Via orchestration
  - Via choreography

Microservices

- Advantages
  - Each MS could use a different technology stack
  - Each MS can be released and deployed independently of others
  - Lower coupling between MSs
- Disadvantages
  - Added complexity
  - Possibly worse time performance

Monolith

- Advantages
  - Simpler initial design and management
- Disadvantages
  - Harder evolution
  - Function scale up more difficult

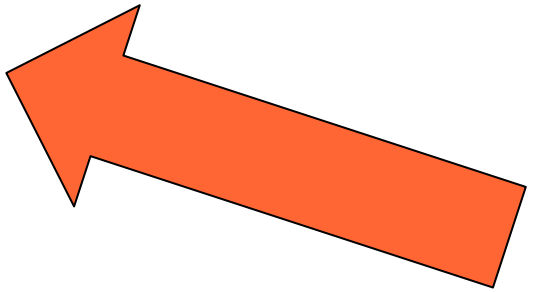
Summary

- Architectural patterns deal with overall system structure
- They provide a unique metaphor for the system (e.g. pipe and filters)
- They address specific domains (e.g. distribution or interaction) and system evolvability

Design

Process

- 1 Architecture
- 2 Design
  - 2.1 High level
  - 2.2 Low level



2.1 Design, high level

- Definition of classes
  - From the glossary: consider a class for each key entity in the glossary
  - From the context diagram:
    - Consider a class for each actor = physical device or subsystem
    - Define GUI for each actor = human actor
- Consider design patterns

2.2 Design, low level

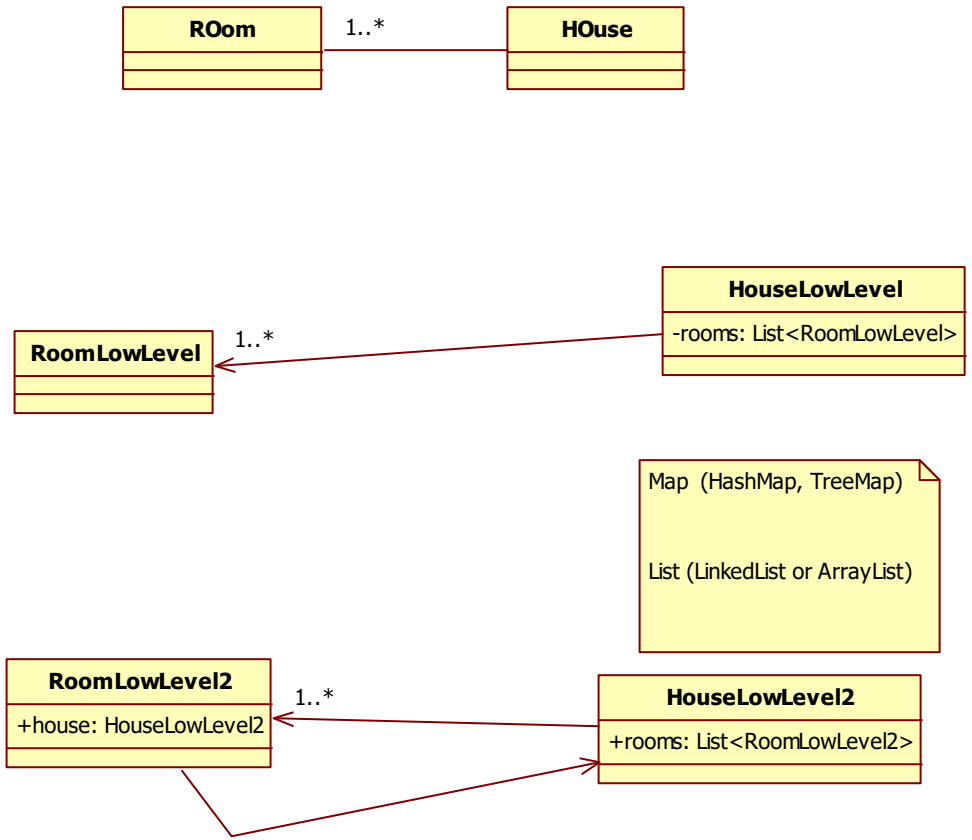
- (inside a class or two)
- For each attribute, define type, privacy
- For each method, define return type, number and type of parameters, and privacy.
- Define setters getters (if needed)
- For each method, choose algorithms (if needed)
- For each relationship with other class, choose the implementation
  - If ‘one’ relationship: reference or key
  - If ‘many’ relationship: array, map, list

2.2 Design, low level

- (inside a class or two)
- Decide persistency
  - No persistence
  - Yes persistence
    - Serialization (to file, to network)
    - To database
      - Decide framework (hybernate, mybatis, slick …) (see ORM lesson)
    - On all objects
    - On part of objects

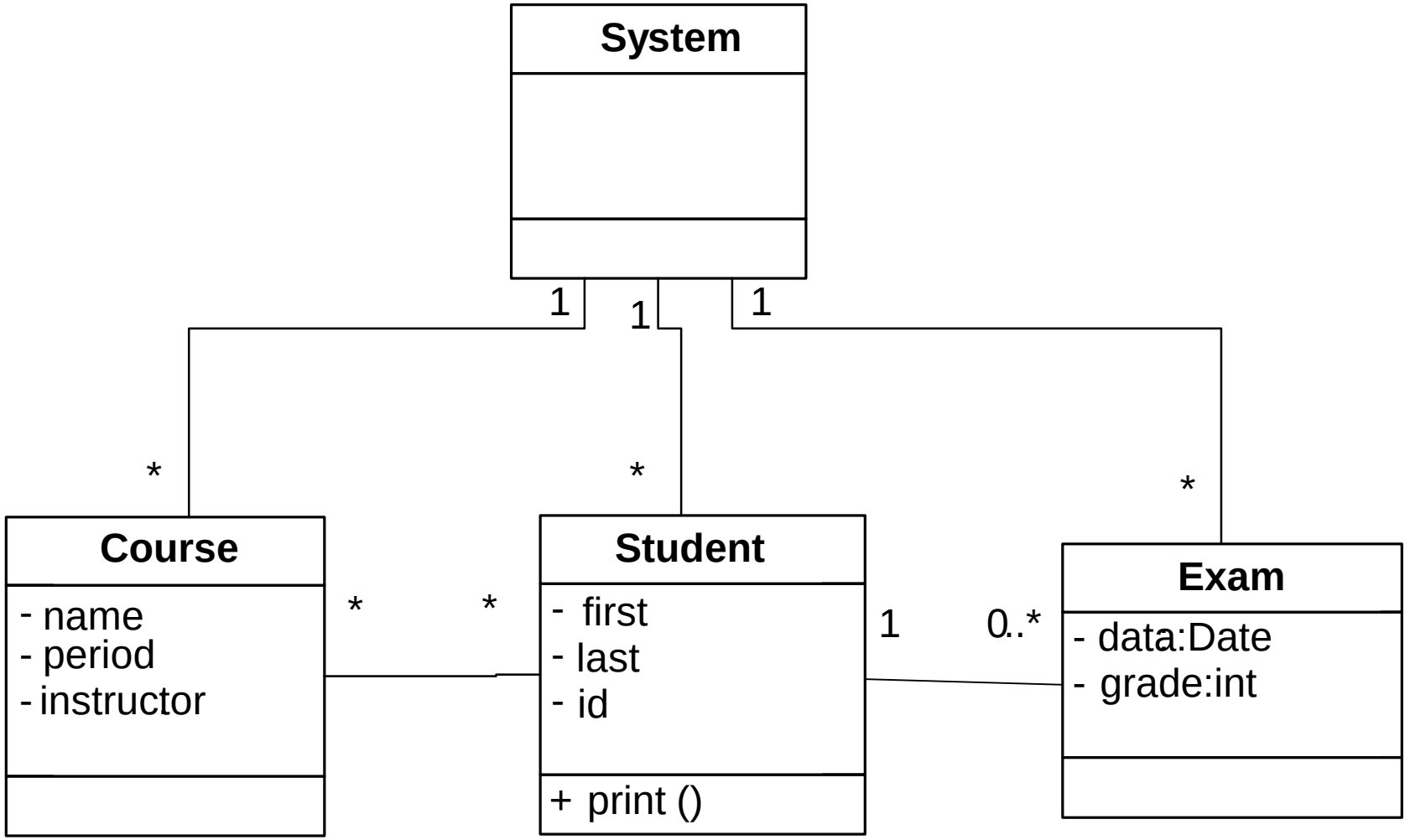
Relationships– low level design

Relationships – 1-1*

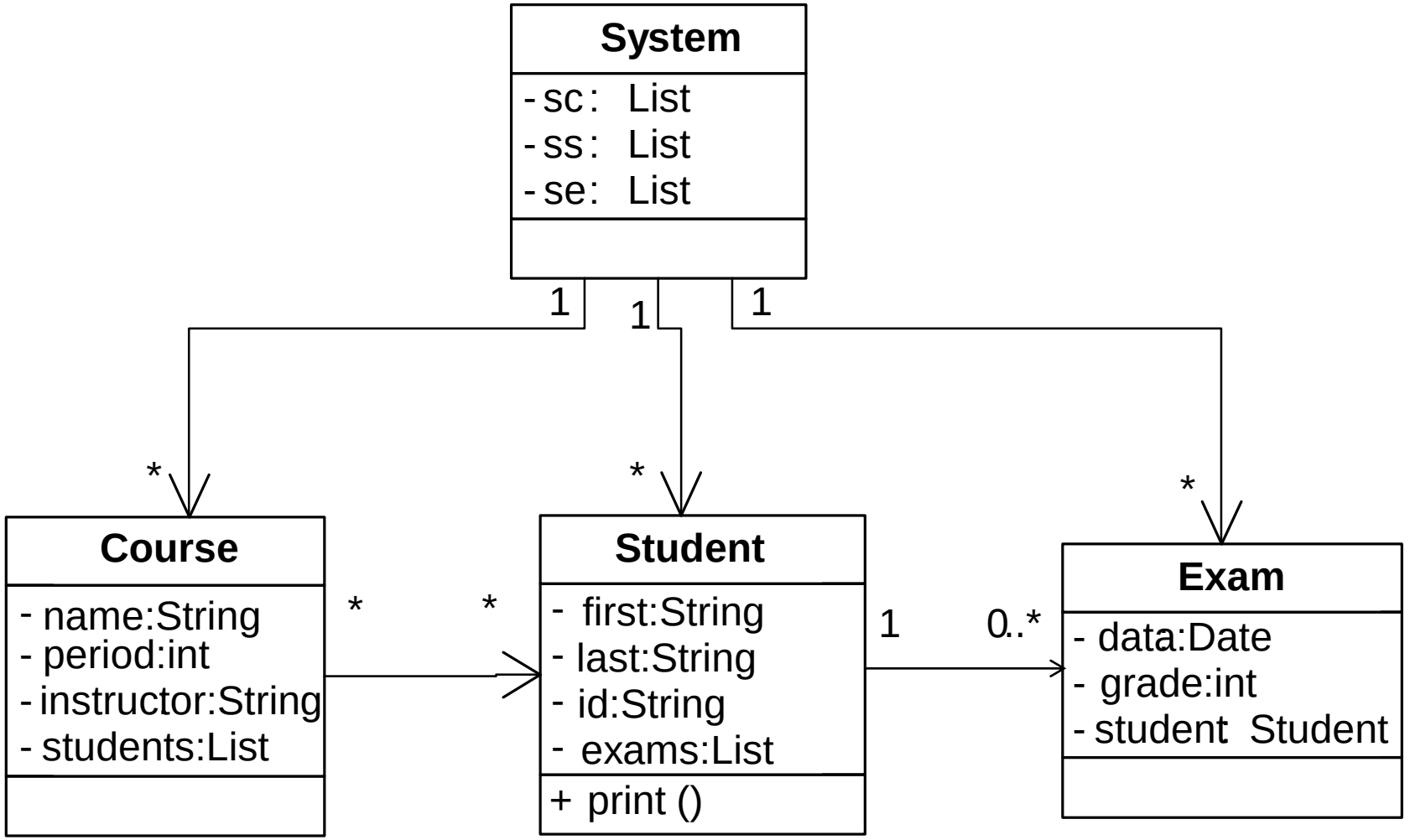


Many to many

Example - glossary

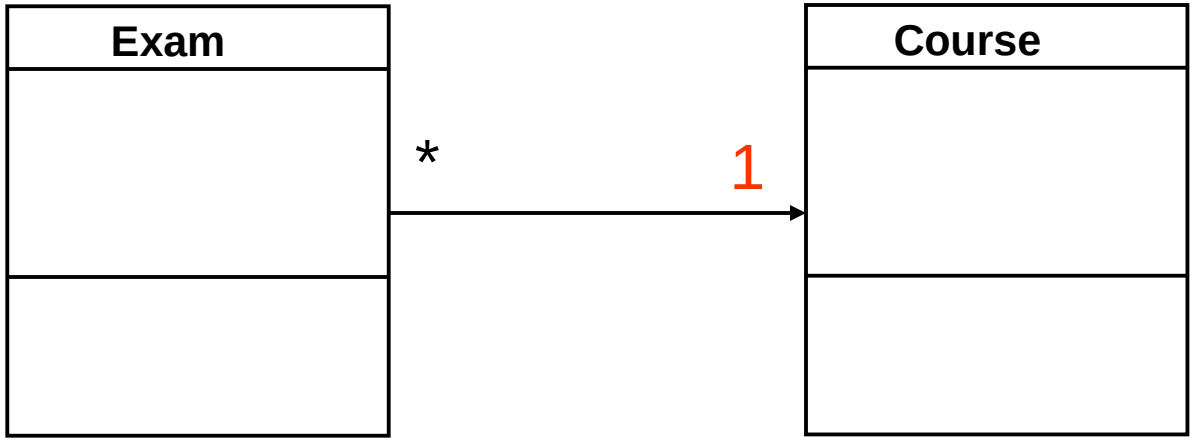


Example



Association :1

- From Exam towards Course

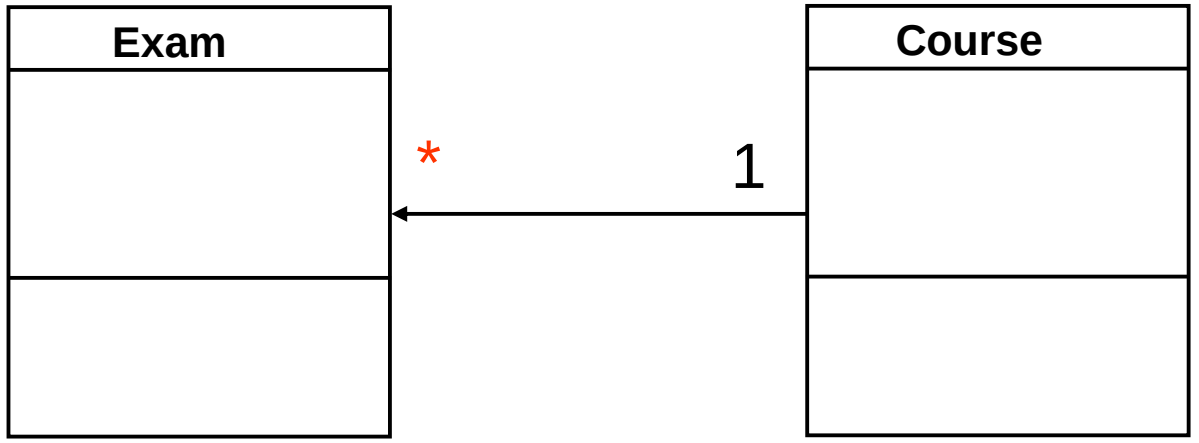



```
class Exam {
    private c: Course;
    setCourse(c: Course): void {
        this.c = c;
    }
}
```

```
class Course {

}
```

Association :n

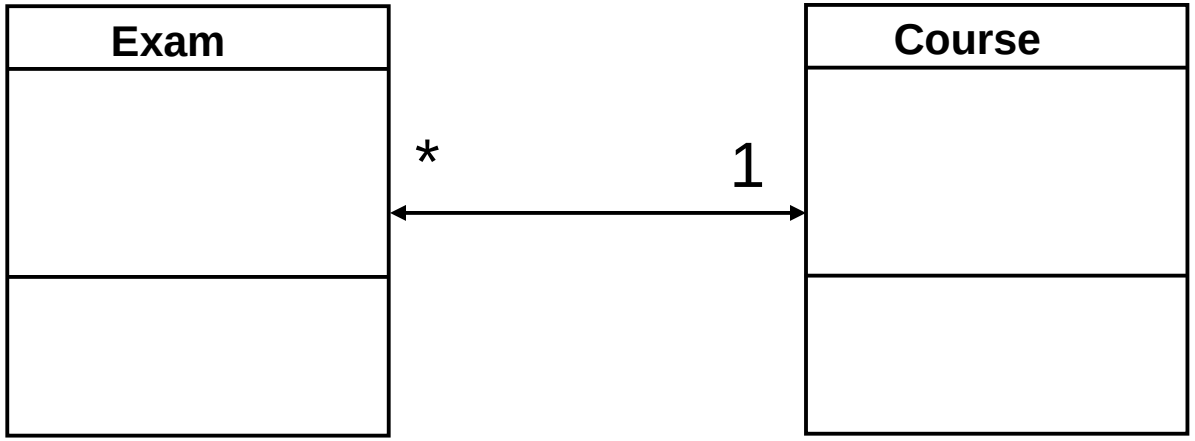
- From Course towards Exams




```
class Course {
    private exams: Exam[];
    constructor() {
        this.exams = [];
    }
    addExam(e: Exam): void {
        this.exams.push(e);
    }
}
```

Association 1:n

- Both directions

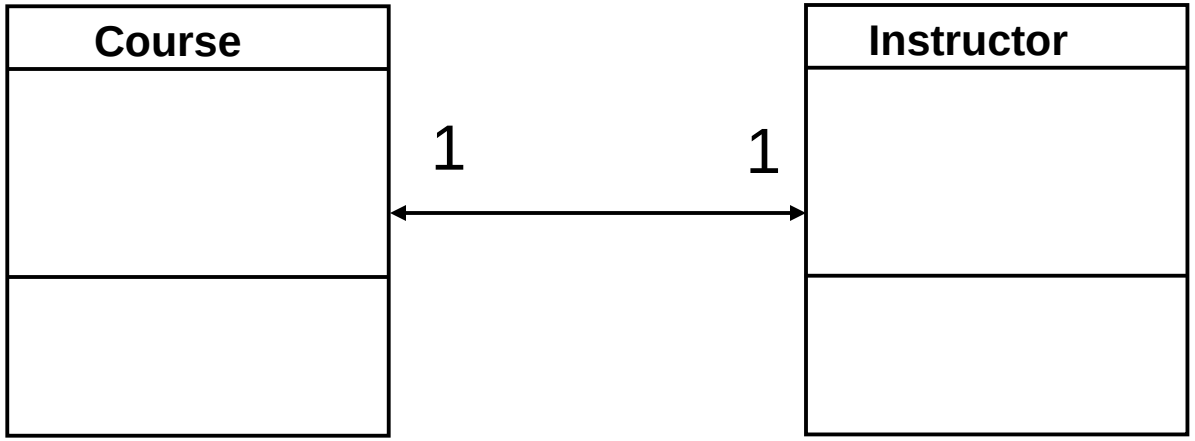



```
class Exam {
    private c: Course;
    setCourse(c: Course): void {
        this.c = c;
    }
}
```

```
class Course {
    private exams: Exam[];
    constructor() {
        this.exams = [];
    }
    addExam(e: Exam): void {
        this.exams.push(e);
    }
}
```

Association 1:1

- Both directions

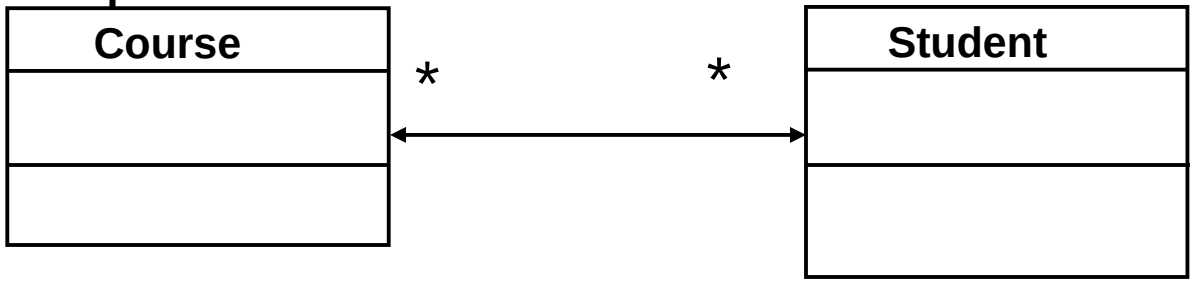



```
class Course {
   private i: Instructor;
}
```

```
class Instructor  {
  private c: Course;
}
```

Association n:m

- Both directions - option1

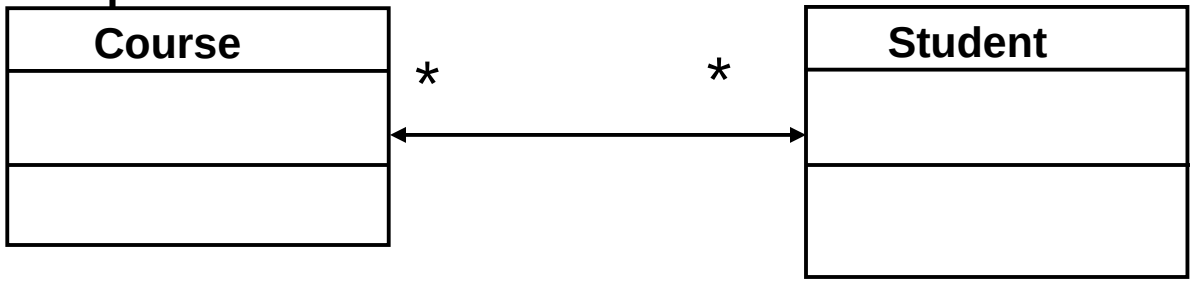



```
class Course {
    private students: Student[];

    constructor() {
        this.students = [];
    }
    addStudent(s: Student): void {
        this.students.push(s);
    }
}
```

```
class Student {
    private courses: Course[];
    constructor() {
        this.courses = [];
    }
    addCourse(c: Course): void {
        this.courses.push(c);
    }
}
```

Association n:m

- Both directions - option 2




```
class Course {
  p: Pair[];

  constructor() {
    this.p = [];
  }
}
```

```
class Student {
  p: Pair[];

  constructor() {
    this.p = [];
  }
}
```

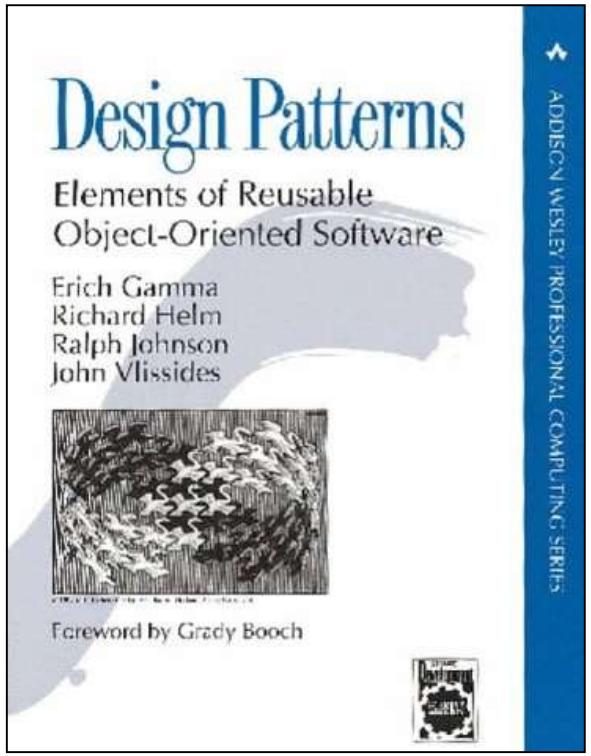
```
class Pair {
  studentKey: number;
  courseKey: number;

  constructor(studentKey: number, courseKey: number) {
    this.studentKey = studentKey;
    this.courseKey = courseKey;
  }
}
```

(Gang of Four) Design Patterns

- Describe the structure of components
- Most widespread category of pattern
- First category of patterns proposed for software development

Design Patterns (GoF)

- Creational
  - E.g. Abstract Factory, Singleton
- Structural
  - E.g. Façade, Composite
- Behavioral
  - *Class:* e.g. Template Method
  - *Object:* e.g. Observer



Design Patterns

- Description of communicating objects and classes that are customized to solve a general design problem in a particular context
- A design pattern names, abstracts, and identifies the key aspects of a common design structure that make it useful for creating a reusable object-oriented design

Description

- Name and classification
- Intent
- Motivation
- Applicability
- Structure
- Participants
- Collaborations

Description

- Consequences
- Implementation
- Sample code
- Known uses
- Related patterns

Classification

- Purpose
  - Creational
  - Structural
  - Behavioral
- Scope
  - Class
  - Object

Pattern selection

- Consider how patterns solve problems
- Scan intent sections
- Study how pattern interrelate
- Study patterns of like purpose
- Examine a cause of redesign
- Consider what should be variable in your design

Using a pattern

- Read through the pattern
- Go back and study
  - Structure
  - Participants
  - Collaborations
- Look at the sample code

Using a pattern

- Choose names for participants
  - Meaningful in the application context
- Define the classes
- Choose operation names
  - Application specific
- Implement operations

Creational patterns

- Factory Method
- Abstract Factory
- Builder
- Prototype
- Singleton

Factory Method

- Delegates the creation of the objects to subclasses.

```typescript
interface Sensor {
  read(): string;
}

class TempSensor implements Sensor {
  read() {
    return "Temperature: 21°C";
  }
}

class HumiditySensor implements Sensor {
  read() {
    return "Umidity: 50%";
  }
}

class SensorFactory {
  static createSensor(type: string): Sensor {
    if (type === "temp") return new TempSensor();
    if (type === "humidity") return new HumiditySensor();
    throw new Error("Sensor not supported");
  }
}


const sensor = SensorFactory.createSensor("temp");
console.log(sensor.read());

```

Abstract Factory

- Context
  - A family of related classes can have different implementation details
- Problem
  - The client should not know anything about which variant they are using / creating

Abstract Factory Example

```typescript
interface Light {
    on(): string;
}
interface Sensor {
    read(): string;
}
interface SmartFactory {
    createLight(): Light;
    createSensor(): Sensor;
}
class HomeLight implements Light {
    on() {
        return "Home light on";
    }
}
class HomeSensor implements Sensor {
    read() {
        return "Home sensor actie";
    }
}
```

```typescript
class HomeFactory implements SmartFactory {
    createLight() {
        return new HomeLight();
    }
    createSensor() {
        return new HomeSensor();
    }
}
class OfficeLight implements Light {
    on() {
        return "Office light on";
    }
}
class OfficeSensor implements Sensor {
    read() {
        return "Office sensor active";
    }
}
class OfficeFactory implements SmartFactory {
    createLight() {
        return new OfficeLight();
    }
    createSensor() {
        return new OfficeSensor();
    }
}
```

```typescript
function setupEnvironment(factory: SmartFactory) {
    const light = factory.createLight();
    const sensor = factory.createSensor();

    console.log(light.on());
    console.log(sensor.read());
}

const factory = process.env.ENV === "office"
    ? new OfficeFactory()
    : new HomeFactory();

setupEnvironment(factory);
```

Builder

```
1   class Scene {
2     private actions: string[] = [];
3
4     addAction(action: string) {
5       this.actions.push(action);
6     }
7
8     run() {
9       return this.actions.join("\n");
10    }
11  }
12
13  class SceneBuilder {
14    private scene = new Scene();
15
16    addLights(): this {
17      this.scene.addAction("Lights on");
18      return this;
19    }
20
```

```
21    addMusic(): this {
22      this.scene.addAction("Play music");
23      return this;
24    }
25
26    addHeating(): this {
27      this.scene.addAction("Heating on");
28      return this;
29    }
30
31    build(): Scene {
32      return this.scene;
33    }
34  }
35
36  const scene = new SceneBuilder()
37      .addLights()
38      .addMusic()
39      .addHeating()
40      .build();
```


Builds the objects step-by-step

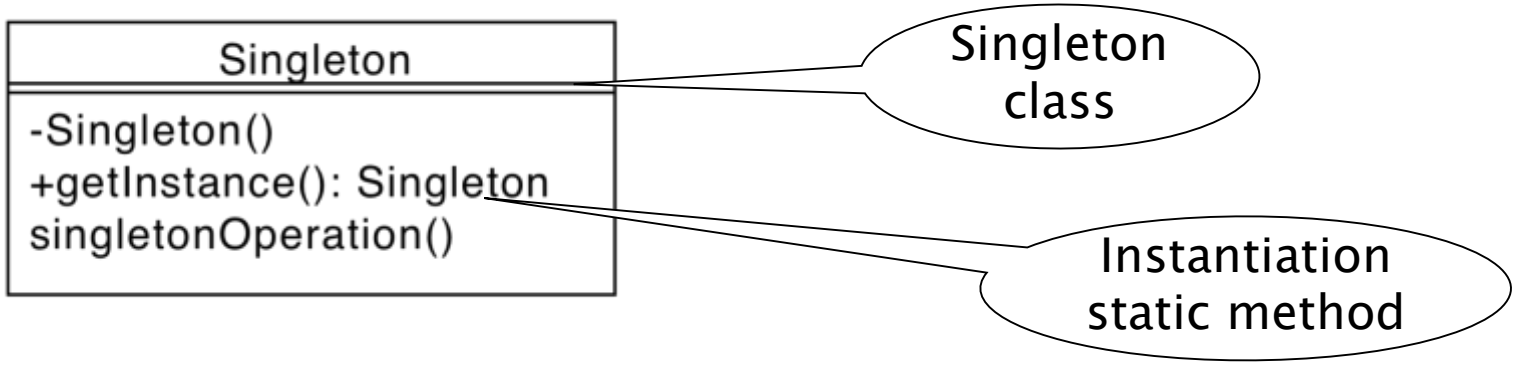
Prototype

```
1   interface Cloneable<T> {
2     clone(): T;
3   }
4
5   class LightConfig implements Cloneable<LightConfig> {
6     constructor(public brightness: number, public color: string) {}
7
8     clone(): LightConfig {
9       return new LightConfig(this.brightness, this.color);
10    }
11  }
12
13  const living = new LightConfig(80, "warm white");
14  const bedroom = living.clone();
15  bedroom.brightness = 40;
16
```

creates new objects by cloning an existing object

Singleton

- Context:
  - A class represents a concept that requires a single instance
- Problem:
  - Clients could use this class in an inappropriate way



Singleton

```
1   class SmartHomeController {
2       private static instance: SmartHomeController;
3
4       private constructor() {} // --> new not possible
5
6       static getInstance(): SmartHomeController {
7         if (!SmartHomeController.instance) {
8           SmartHomeController.instance = new SmartHomeController();
9         }
10        return SmartHomeController.instance;
11      }
12
13      status() {
14        return "Control room";
15      }
16    }
17
18    const ctrl1 = SmartHomeController.getInstance();
19    const ctrl2 = SmartHomeController.getInstance();
20    console.log(ctrl1 === ctrl2); // --> true
```

Structural patterns

- Structural patterns are concerned with how classes and objects are composed to form larger structures.

GoF structural patterns

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Adapter

- Context:
  - A class provides the required features, but its interface is not the one required
- Problem:
  - How is it possible to integrate the class without modifying it
    - Its source code could be not available
    - It is already used as it is somewhere else

Adapter

Inheritance plays a fundamental role

```
1  // Existing class
2  class OldSensor {
3      getValue(): number { return 42; }
4  }
5
6  // New expected interface
7  interface Sensor {
8    read(): string;
9  }
10
11 // Adapter
12 class SensorAdapter implements Sensor {
13   constructor(private oldSensor: OldSensor) {}
14   read(): string {
15     return `Reading: ${this.oldSensor.getValue()} units`;
16   }
17 }
18
19 // Usage
20 const sensor: Sensor = new SensorAdapter(new OldSensor());
21 console.log(sensor.read());
22
```

Bridge

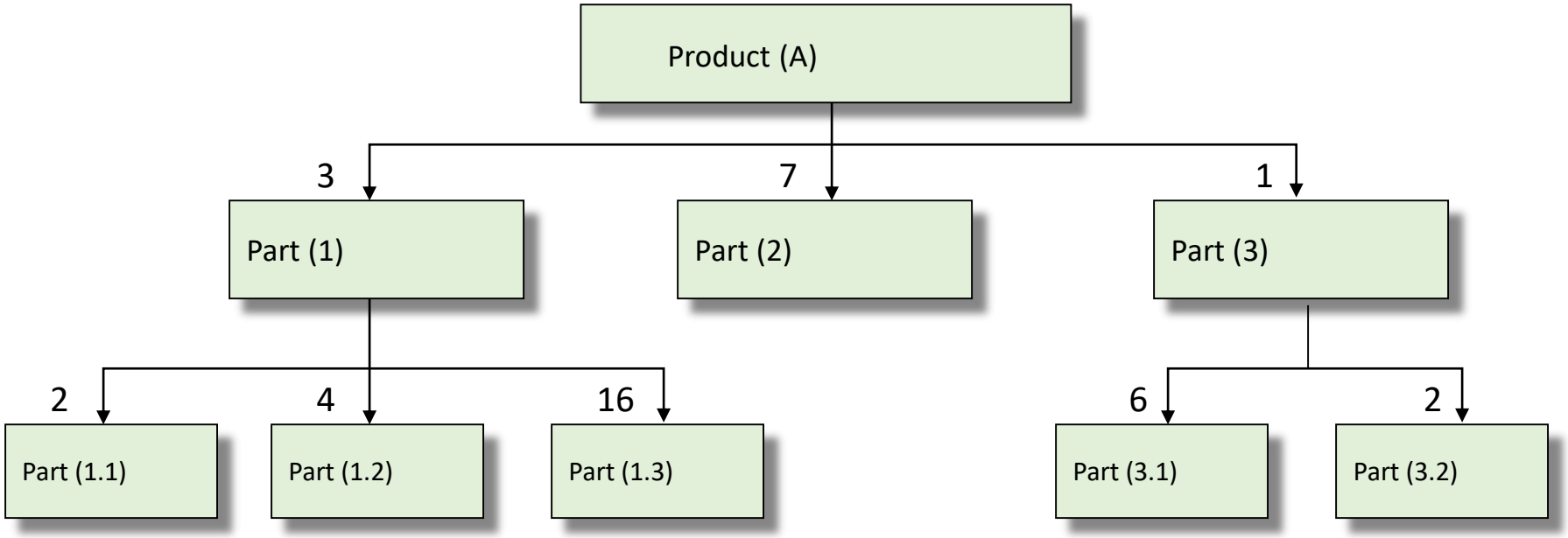
- Separates abstraction from implementation.

```
1   interface Device {
2     turnOn(): string;
3   }
4
5   class TV implements Device {
6     turnOn() {
7       return "TV is now ON";
8     }
9   }
10
11  class Remote {
12    constructor(protected device: Device) {}
13    pressPower() {
14      return this.device.turnOn();
15    }
16  }
17
18  const remote = new Remote(new TV());
19  console.log(remote.pressPower());
20
```

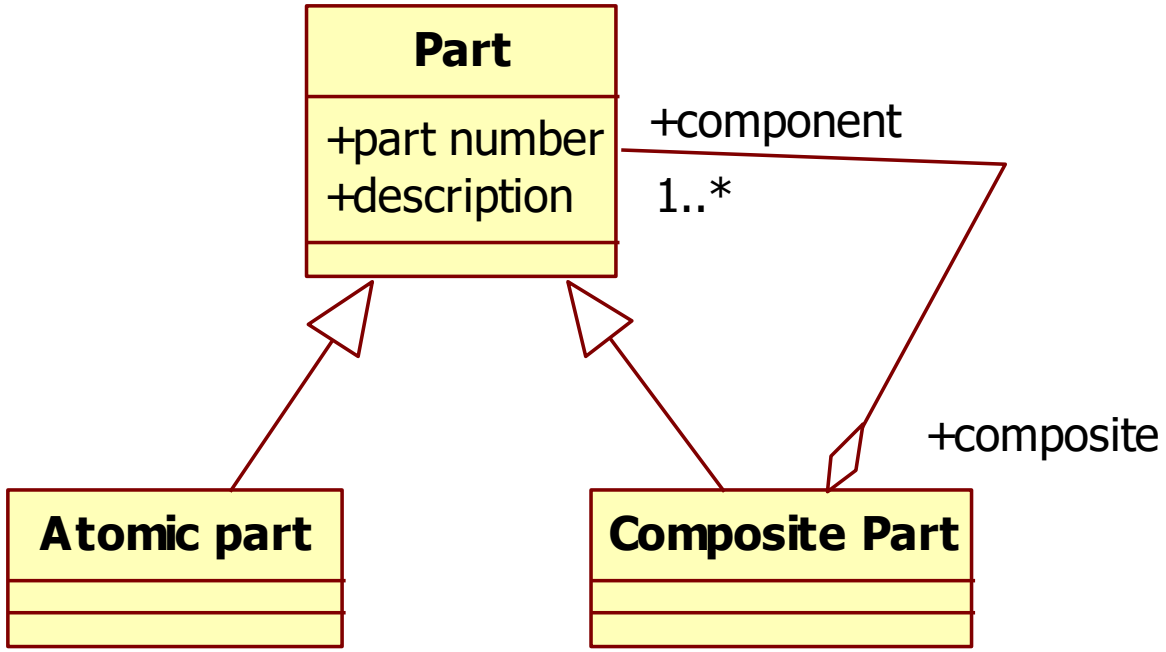
Composite

- Context:
  - You need to represent part-whole hierarchies of objects
- Problem
  - Clients are complex
  - Difference between composition objects and individual objects.

Bill of materials (BOM)



Composite example: BOM

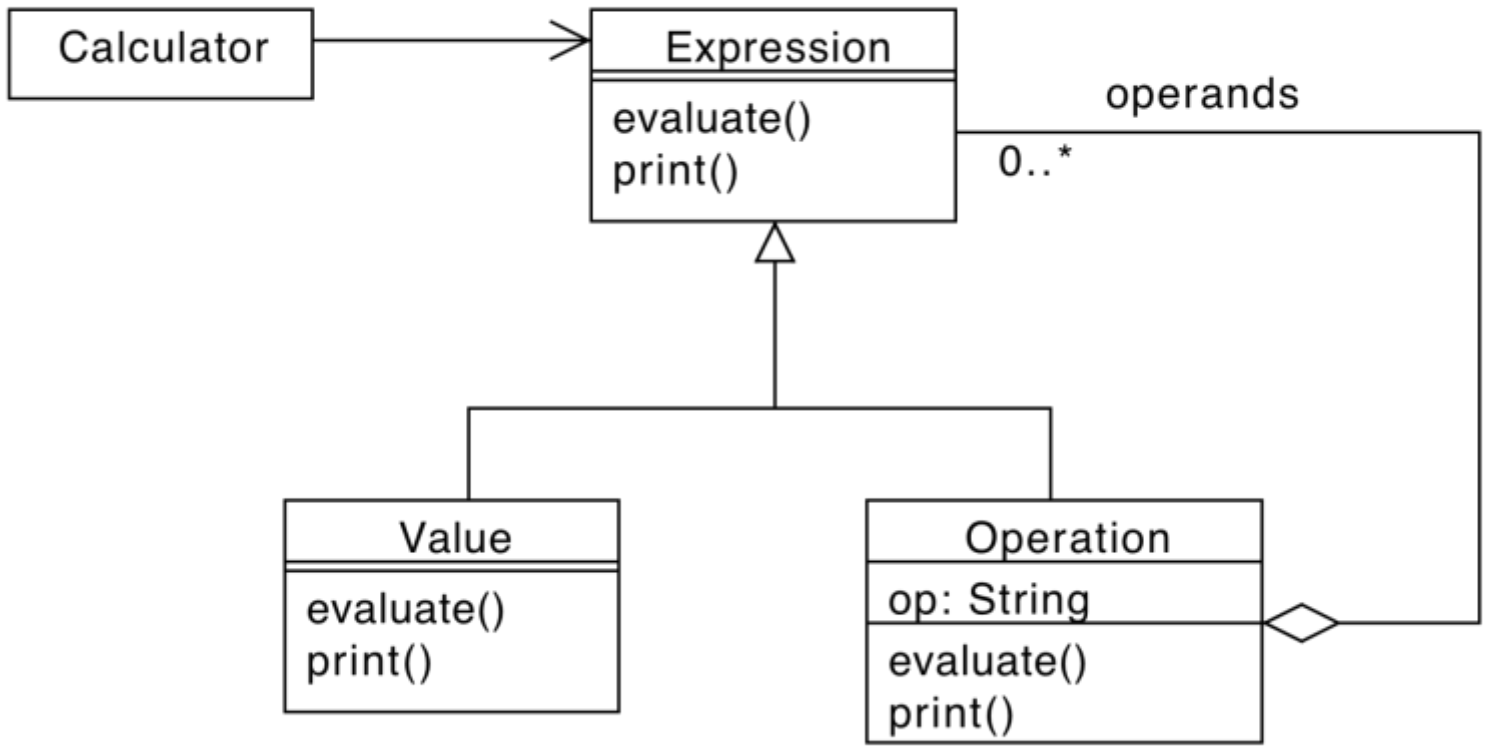


Composite Example

- Arithmetic expressions representation
  - Operators
  - Operands

  - A+ B * (A + B)
- Evaluation of expressions

Composite Example



Composite Example

```
1   abstract class Expression {
2       public abstract evaluate(): number;
3       public abstract print(): string;
4   }
5
6   class Value extends Expression {
7       private value: number;
8       constructor(v: number) {
9           super();
10          this.value = v;
11      }
12      public evaluate(): number {
13          return this.value;
14      }
15      public print(): string {
16          return this.value.toString();
17      }
18  }
19
```

Composite Example

```
20  class Operation extends Expression {
21      private op: string; // '+' | '-' | '*' | '/'
22      private left: Expression;
23      private right: Expression;
24
25      constructor(op: string, left: Expression, right: Expression) {
26          super();
27          this.op = op;
28          this.left = left;
29          this.right = right;
30      }
31      public evaluate(): number {
32          switch (this.op) {
33              case '+':
34                  return this.left.evaluate() + this.right.evaluate();
35              case '-':
36                  return this.left.evaluate() - this.right.evaluate();
```

```
37              case '*':
38                  return this.left.evaluate() * this.right.evaluate();
39              case '/':
40                  const rightEval = this.right.evaluate();
41                  if (rightEval === 0) {
42                      throw new Error("Division by zero");
43                  }
44                  return this.left.evaluate() / rightEval;
45              default:
46                  throw new Error("Unsupported operation");
47          }
48      }
49      public print(): string {
50          return `(${this.left.print()} ${this.op}
51                      ${this.right.print()})`;
52      }
53  }
```

Decorator

- Context:
  - You need to add responsibilities to individual objects dynamically and transparently, without affecting other objects.
- Problem
  - You want to avoid creating many subclasses for every possible combination of features.
  - You need flexibility to stack features dynamically at runtime.
  - Inheritance is too rigid: extending a class for every combination leads to class explosion.

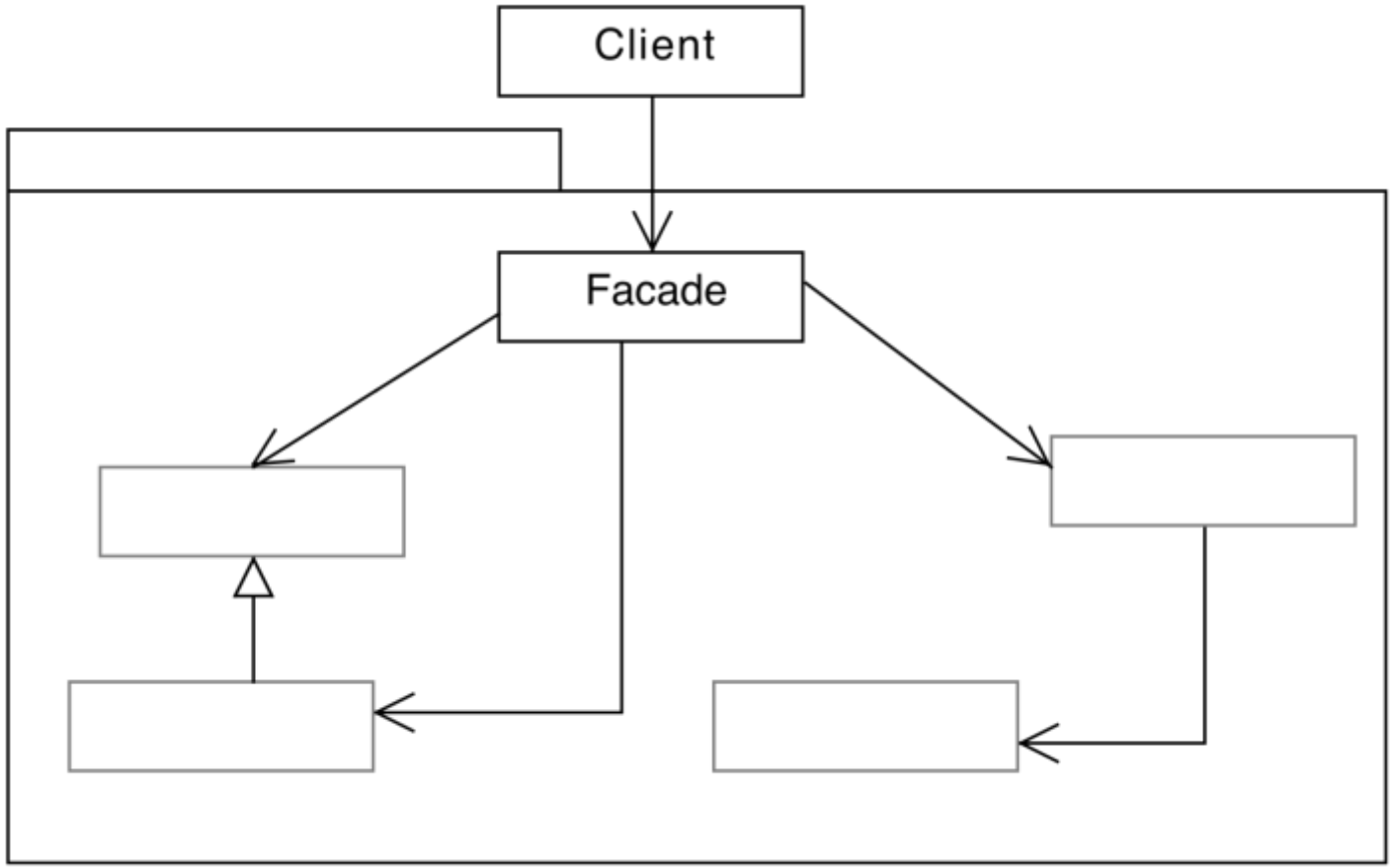
Decorator Example

```
1   interface Message {
2       getContent(): string;
3   }
4
5   class TextMessage implements Message {
6     getContent() { return "Hello"; }
7   }
8
9   class EncryptedMessage implements Message {
10    constructor(private message: Message) {}
11    getContent() {
12      return this.encrypt(this.message.getContent());
13    }
14    encrypt(content: string): string {/*encription function*/ return content;}
15  }
16
17  const message = new EncryptedMessage(new TextMessage());
18  console.log(message.getContent());
```

Façade

- Context
  - A functionality is provided by a complex group of classes (interfaces, associations, etc.)
- Problem
  - How is it possible to use the classes without being exposed to the details

Façade



Façade

```
1   class A {
2       public method1(): void { }
3   }
4   class B {
5       public method2(): void { }
6   }
7   class C {
8       public method3(): void { }
9   }
10
```

Without Façade

- Client

```
a: A; b: B; c: C;
a.method1();
b.method2();
c.method3();
```

With facade

```
12  class Facade {
13      private a: A = new A();
14      private b: B = new B();
15      private c: C = new C();
16      public method1(): void {
17          this.a.method1();
18      }
19
20      public method2(): void {
21          this.b.method2();
22      }
23      public method3(): void {
24          this.c.method3();
25      }
26  }
27
28  const facade = new Facade();
29  facade.method1();
30  facade.method2();
31  facade.method3();
```

Flyweight

- Context
  - Need to support a large number of fine-grained objects efficiently.
- Problem
  - Creating thousands (or millions) of similar objects consumes much memory.
  - Many objects share the same data, but you are duplicating it.
  - You want to minimize memory usage by sharing as much data as possible.

Flyweight

```
1   class Icon {
2     constructor(private type: string) {}
3     draw(x: number, y: number) {
4       return `Draw ${this.type} at (${x}, ${y})`;
5     }
6   }
7
8   class IconFactory {
9     private icons: Record<string, Icon> = {};
10    getIcon(type: string) {
11      if (!this.icons[type]) this.icons[type] = new Icon(type);
12      return this.icons[type];
13    }
14  }
15
16  const factory = new IconFactory();
17  console.log(factory.getIcon("user").draw(10, 20));
18  console.log(factory.getIcon("user").draw(30, 40));
19  console.log(factory.getIcon("admin").draw(50, 60));
20
```

Proxy

- Context
  - You need to control access to an object, add behavior before or after calls, or defer expensive object creation.
- Problem
  - Direct access to some objects can be inefficient, unsafe, or undesirable.
  - Need to:
    - Add lazy initialization
    - Implement access control
    - Log or monitor interactions
    - Protect from remote or expensive operations

Proxy

```typescript
interface DataService {
  fetch(): string;
}

class RealService implements DataService {
  fetch() {
    return "Sensitive data from real service";
  }
}

class ProxyService implements DataService {
  constructor(private real: RealService, private isAuthorized: boolean) {}

  fetch() {
    if (!this.isAuthorized) {
      return "Access denied";
    }
    return this.real.fetch();
  }
}

const proxy = new ProxyService(new RealService(), false);
// Access denied

const secureProxy = new ProxyService(new RealService(), true);
// Data
```

Behavioral patterns

- Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects.
- Not just patterns of objects or classes but also the patterns of communication.
  - Complex control flow that's difficult to follow at run-time.
  - Shift focus away from flow of control to let concentrate just on the way objects are interconnected.

GoF behavioral patterns

- Object-level
  - Chain of Responsibility
  - Command
  - Iterator
  - Mediator
  - Memento
  - Observer
  - State
  - Strategy
  - Visitor
- Class-level
  - Template Method
  - Interpreter

Behavioral Patterns

Pattern	Intent	When to Use	Key Characteristics
Chain of Responsibility	Pass a request along a chain of handlers	When you want to decouple the sender of a request from its potential receivers	Dynamic chain, flexible processing, receiver selection at runtime
Command	Encapsulate a request as an object	When you need undo/redo, queueing, or logging of operations	Command -> Receiver -> execute(), supports macros and queuing
Iterator	Provide a standard way to traverse elements in a collection	When clients need access to elements without exposing the internal structure	Uniform access to different collection types
Mediator	Centralize communication between components	When many objects communicate in complex, tightly coupled ways	Reduces dependencies among colleagues, promotes loose coupling

Object – level Behavioral Patterns

Pattern	Intent	When to Use	Key Characteristics
Memento	Capture and restore an object's state without violating encapsulation	When you need undo functionality or checkpoints	Uses a Caretaker to store internal state snapshots
Observer	Notify dependent objects automatically when state changes	When changes in one object need to propagate to many observers	Publisher/subscriber, supports event-driven systems
State	Alter object behavior when its internal state changes	When an object's behavior depends on its state	Replaces conditional logic with polymorphism
Strategy	Encapsulate interchangeable algorithms	When you want to switch between multiple behaviors dynamically	Promotes reuse, clean separation of concerns
Visitor	Separate algorithms from object structure	When you want to perform operations on elements of a complex object structure (e.g., AST, file tree)	Double dispatch, new operations without modifying classes

Class – level Behavioral Patterns

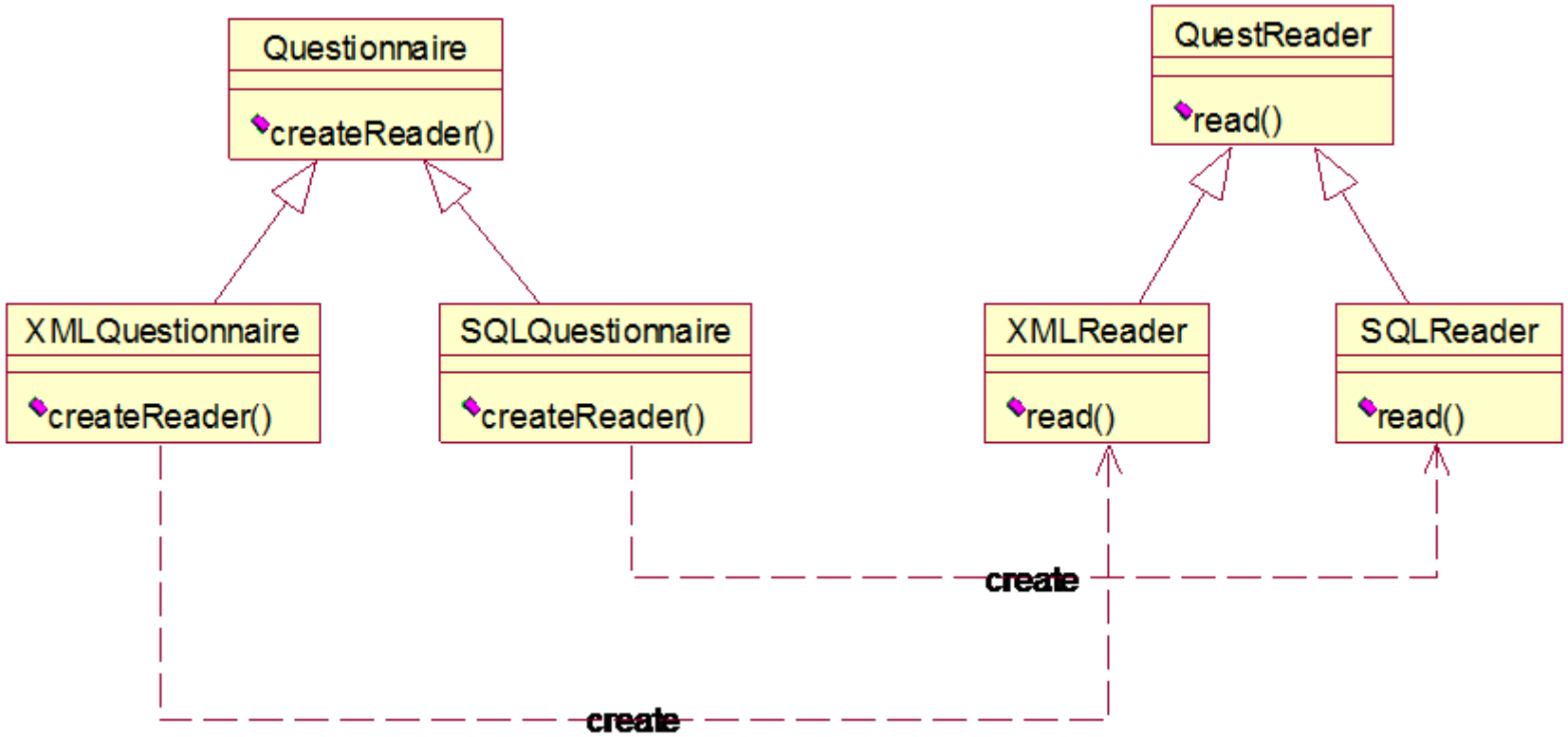
Pattern	Intent	When to Use	Key Characteristics
Template Method	Define the skeleton of an algorithm, deferring steps to subclasses	When you have a general algorithm structure with variation in specific steps	Partially implemented base class, uses abstract or hook methods
Interpreter	Define a grammar and interpreter for language expressions	When you need to interpret expressions from a grammar (e.g., rules, formulas)	AST + grammar rules, used in DSLs and parsing

Analysis With Patterns

- Process:
  - Find out what patterns are used
  - Find out what the role assignments are
  - Find out how functionalities are implemented using patterns
  - …use this knowledge

- https://github.com/torokmark/design_patterns_in_typescript

Exercise – Data Access



Exercise - Questionnaire

```
interface QuestReader {
    read(): void;
}
abstract class Questionnaire {
    private static single: Questionnaire;
    public static getQuestionnaire(): Questionnaire {
        if (!Questionnaire.single) {
            Questionnaire.single = new
  ConcreteQuestionnaire();
        }
        return Questionnaire.single;
    }
    public abstract createReader(): QuestReader;
}
```

Exercise - Questionnaire

```
class ConcreteQuestionnaire extends Questionnaire {
    public createReader(): QuestReader {
        return new ConcreteQuestReader();
    }
}

class ConcreteQuestReader implements QuestReader {
    public read(): void {
        console.log("Reading questions...");
    }
}
```

Exercise

- What patterns are used in this example?
- What are the role assignments?
- What purpose do(es) the pattern(s) serve?

Verification

Verification

- Functional requirements
  - Traceability matrix
  - Scenarios executed on architecture
  - Inspection
- Non functional requirements
  - Performance
    - Scenarios enriched with time model
  - (Inspection)

Traceability matrix

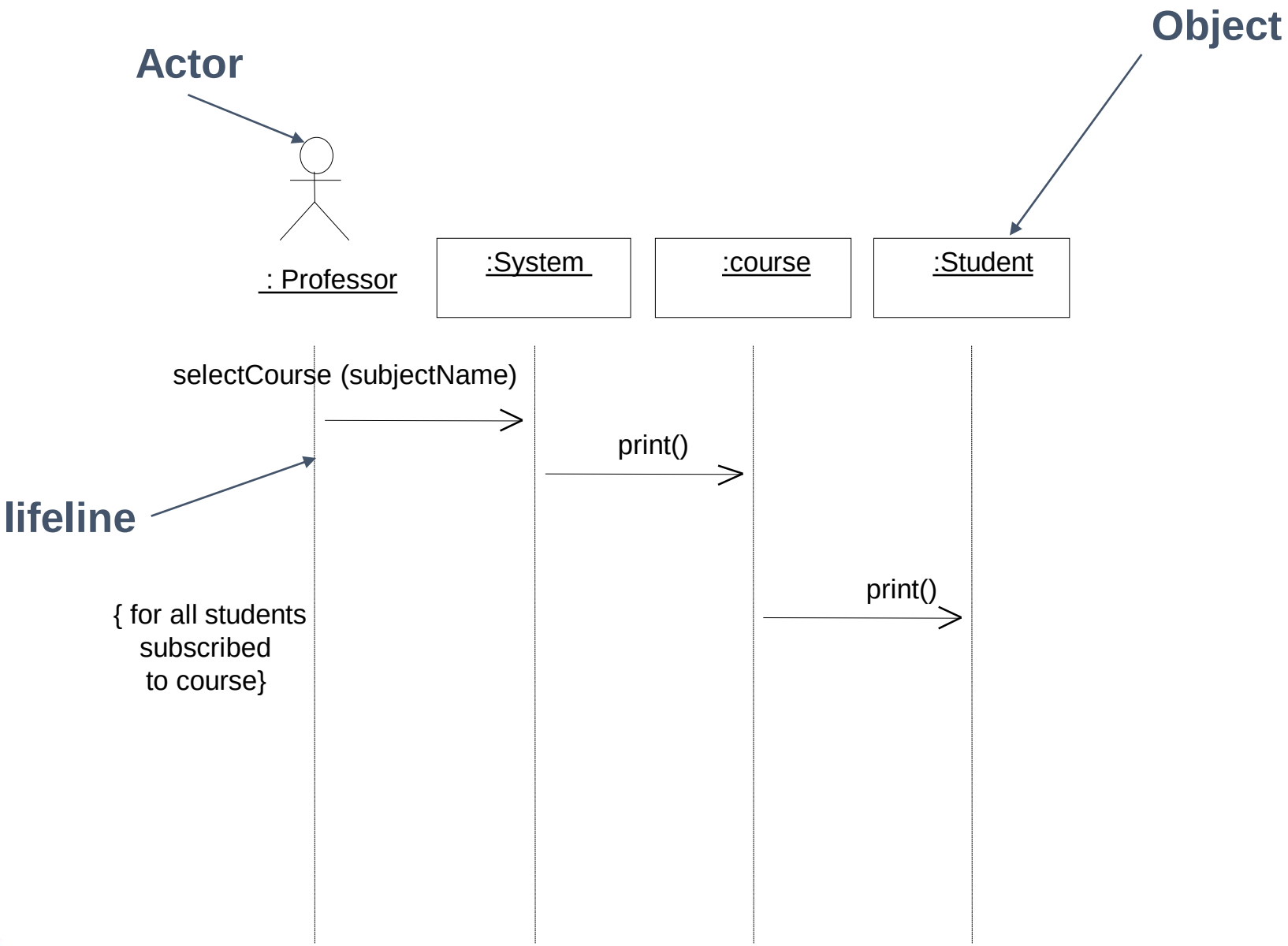
	AwayManagementStrategy	Boiler	CRoom	DefaultHouseSettings	Env	Environment	HouseController	InvalidTimeException	PhysBoiler	PresenceManagementStrategy	Room	RoomManagementStrategy	RoomSettings	SetRoomParametersActivity	SetRoomParametersDialog	XMLSettings
Temp-UR-F1											X		X	X	X	X
Temp-UR-F2											X		X	X	X	X
Temp-UR-F3											X		X	X	X	X
Temp-UR-F4											X		X	X	X	X
Temp-UR-F5											X		X	X	X	X
Temp-UR-F6		X	X		X	X	X		X	X	X	X				
Temp-UR-F7	X	X	X		X	X	X		X		X	X				
Temp-UR-F8		X	X		X	X	X		X	X	X	X				
Temp-UR-F9		X	X		X	X	X		X	X	X	X				
Temp-UR-F10	X	X	X		X	X	X		X		X	X				
Temp-UR-F11								X							X	
Temp-UR-F12				X									X	X		
Temp-UR-F13	X	X	X		X	X	X		X		X	X				
Temp-UR-F14	X		X		X	X	X			X	X	X				
Temp-UR-F15			X		X	X	X			X	X	X				
Temp-UR-F16	X									X						
Temp-UR-F17			X	X			X				X					X
Temp-UR-F18		X					X		X							
UR-Inv 1	X	X	X		X	X	X		X		X	X				
UR-Inv 2		X	X		X	X	X		X	X	X	X				

Traceability matrix

- Each functional requirement (from requirements document) must be supported by at least one function in one class in the software design
  - The more complex the requirement, the more member functions needed

Scenarios

- Each scenario (from requirements document) must be feasible
  - It is possible to define a sequence of calls to member functions of classes in the software design that matches the scenario



Key points

- Architecture
  - defining high level components and their control, communication model
  - Tools: UML or ADL models, structural and dynamic
  - Styles: Layered, client server (2 tier, 3 tier), peer to peer, shared repository
- Design
  - Define internals of components
  - Tools: UML models
  - Design patterns

Key points

- Verification
  - inspections
    - Architecture can satisfy functional properties (as defined in requirements doc)?
      - Traceability matrixes
      - Scenario execution
    - Architecture can satisfy non functional requirements?
      - Enriched scenarios
  - build prototype

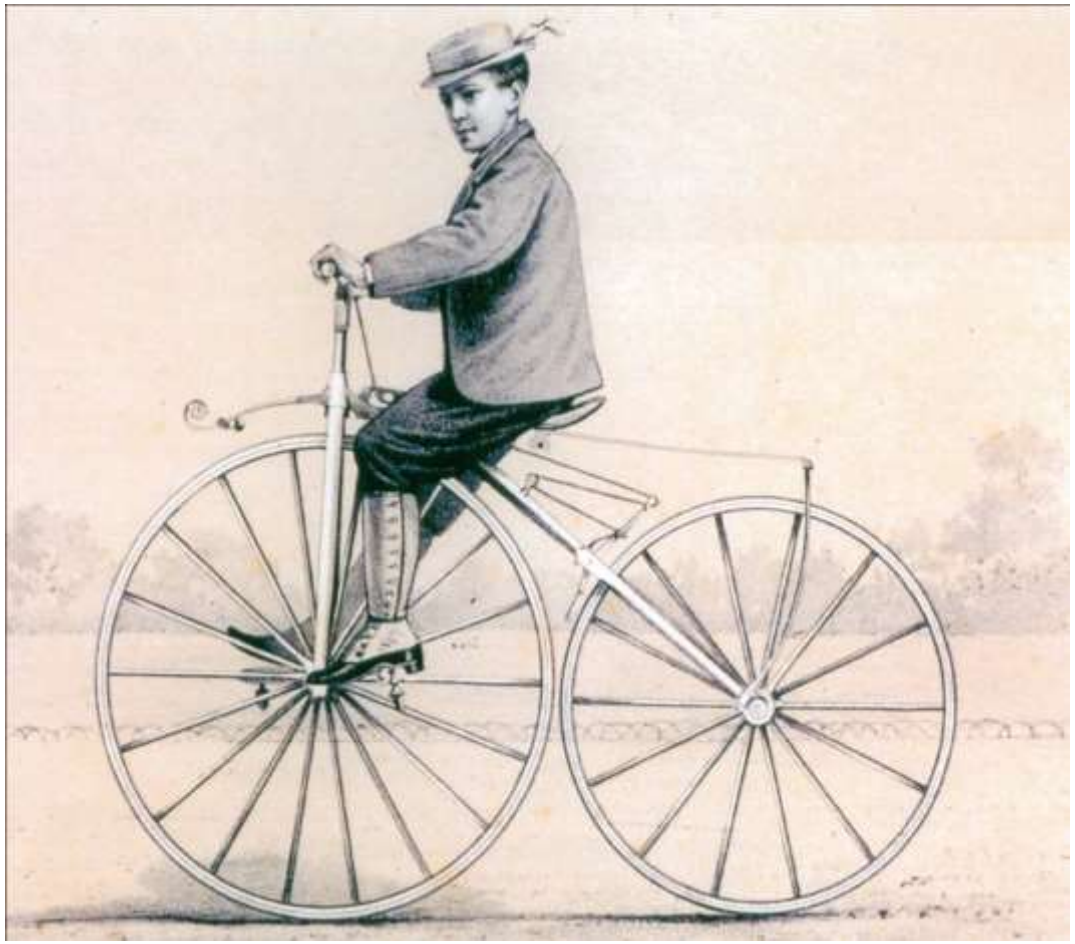
Bicycles ..

Draisine

- 1820
- Front wheel steering
- Foot powered

Velocipede

- 1860
- Front wheel steering
- Crank pedal on front wheel



Penny farthing

- 1870
- Larger front wheel
  - More speed
  - More comfort
  - unstable

Dwarf ordinary

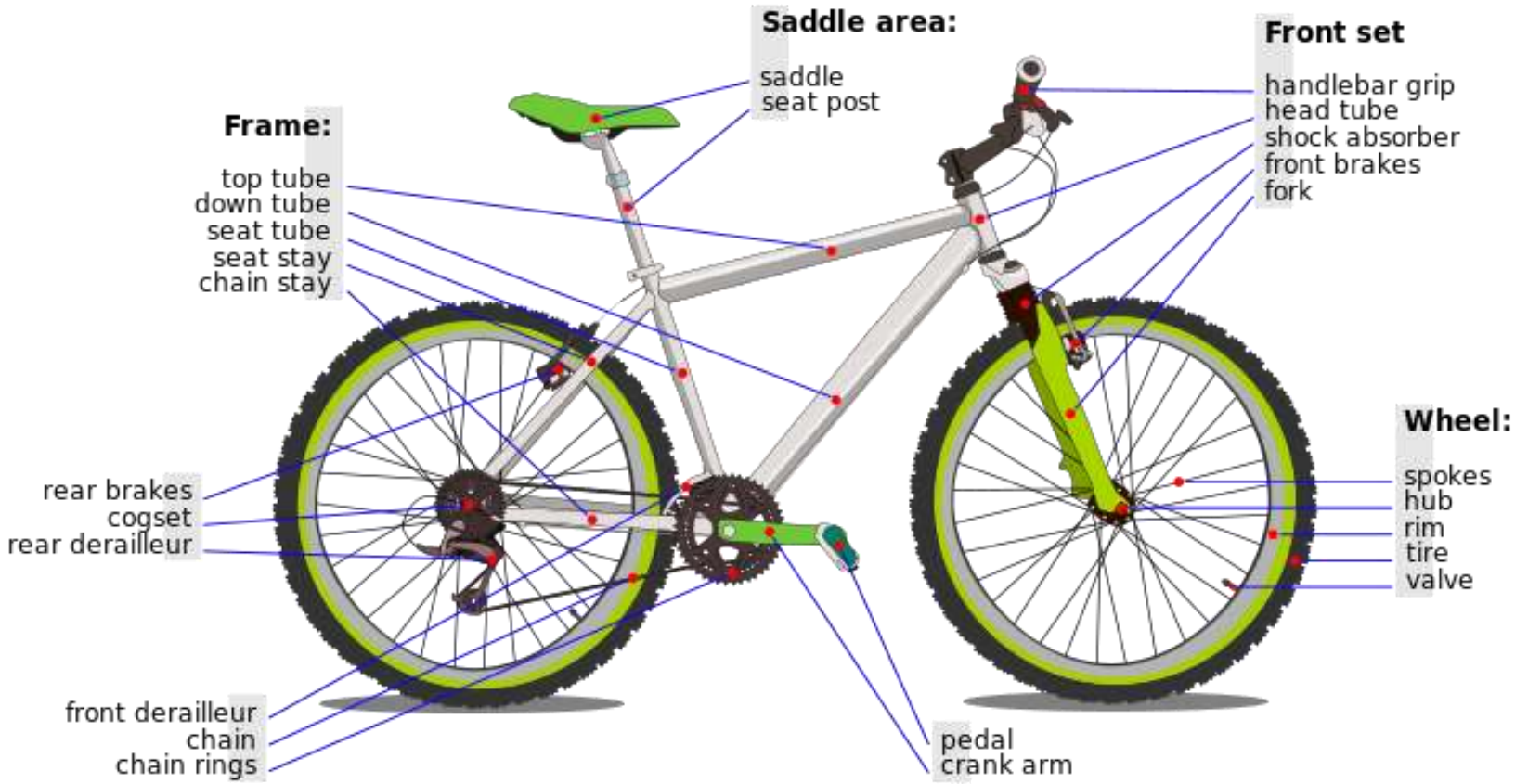
- Smaller front wheel, seat backwards
- More stable, less speed, less confort

THOMAS MCCALL AND HIS BICYCLE.
(From a Photograph by Bruce and Howie, of Kilmarnock.)

And ..

- 1870, chain drive
  - Solves problem of steering and pedaling on front wheel
  - Pedals in middle, power to rear wheel
- 1885, seat tube (diamond frame)
- 1888, pneumatic tire (Dunlop)
  - Comfort
- 1890
  - Rear freewheel (coasting)
- 1905
  - Derailleur gears

Dominant design



Dominant design

- Requires time to develop and be commonly shared in the domain
- Requires specific components
- Leads to specialized companies / roles
  - Company to design/develop tyres
  - Company to design/develop chains
  - Etc..

Other designs

Requirements - bike

- Functional requirements
  - transport one person from place to place
    - Steer
    - accelerate
    - brake
- Non functional requirements
  - Efficiency : speed from 10 km/h to 50 km/h
    - (Speed from 10 km/h to 150 km/h)
  - Efficiency : weight between 10 and 15kg
  - Efficiency: reasonable torque to start: < 40Nmeters
  - Usability: out of 50 average users, at least 60% of them can use it within 10 minutes
  - Only human power (no engines)
  - Safety (no harm to driver)
  - Security (difficult to steal)
  - Cost (between 100 and 200 euro)

Design vs requirements

	Draisine	Velocipede	Penny farthing	Another design	Dominant design
Transport one person	y	Y	Y	Y	Y
Eff – speed	< 10kmh	Y	Y	Y	Y
Eff – torque at start	Y	N	N	Y	Y
Eff – weight	y	Y	Y	Y	y
Human power	y	Y	Y	Y	Y
safety	Driver less high	Driver vey high	Driver even higher	Y	Y
Reduce speed	With feet on road	Applying negative force to pedal	Applying negative force to pedal	Y	y brakes